

About Jamboree Education

Jamboree has helped thousands of students like you make it to top colleges abroad. Be it GMAT, GRE or SAT, their unique problem-solving methods ensure maximum scores with minimum effort. They recently launched a feature where students/learners can come to their website and check their probability of getting into the IVY league college. This feature estimates the chances of graduate admission from an Indian perspective.

Column Profiling

Serial No. (Unique row ID)

GRE Scores (out of 340)

TOEFL Scores (out of 120)

University Rating (out of 5)

Statement of Purpose and Letter of Recommendation Strength (out of 5)

Undergraduate GPA (out of 10)

Research Experience (either 0 or 1)

Chance of Admit (ranging from 0 to 1)

Problem Statement

Identify and analyze the most significant features that impact admission decisions.

Develop a predictive model that can estimate an applicant's likelihood of admission based on their academic and profile-related attributes.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from statsmodels.stats.outliers_influence import
variance_inflation_factor
from sklearn.metrics import r2_score
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
from scipy import stats
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso
from sklearn.linear_model import ElasticNet
import warnings
warnings.filterwarnings('ignore')
```

Loading Data Set and Performing EDA

```
df = pd.read_csv('Jamboree_Admission.csv')
df.head()
```

```
{
  "summary": {
    "name": "df",
    "rows": 500,
    "fields": [
      {
        "column": "Serial No.",
        "properties": {
          "dtype": "number",
          "std": 144,
          "min": 1,
          "max": 500,
          "num_unique_values": 500,
          "samples": [
            362, 74, 375
          ],
          "semantic_type": ""
        },
        "description": "",
        "column": "GRE Score",
        "properties": {
          "dtype": "number",
          "std": 11,
          "min": 290,
          "max": 340,
          "num_unique_values": 49,
          "samples": [
            307, 335, 297
          ],
          "semantic_type": ""
        },
        "description": "",
        "column": "TOEFL Score",
        "properties": {
          "dtype": "number",
          "std": 6,
          "min": 92,
          "max": 120,
          "num_unique_values": 29,
          "samples": [
            94, 119, 112
          ],
          "semantic_type": ""
        },
        "description": "",
        "column": "University Rating",
        "properties": {
          "dtype": "number",
          "std": 1,
          "min": 1,
          "max": 5,
          "num_unique_values": 5,
          "samples": [
            3, 1, 2
          ],
          "semantic_type": ""
        },
        "description": "",
        "column": "SOP",
        "properties": {
          "dtype": "number",
          "std": 0.9910036207566069,
          "min": 1.0,
          "max": 5.0,
          "num_unique_values": 9,
          "samples": [
            1.0, 4.0, 5.0
          ],
          "semantic_type": ""
        },
        "description": "",
        "column": "LOR",
        "properties": {
          "dtype": "number",
          "std": 0.9254495738978181,
          "min": 1.0,
          "max": 5.0,
          "num_unique_values": 9,
          "samples": [
            3.5, 1.5, 5.0
          ],
          "semantic_type": ""
        },
        "description": "",
        "column": "CGPA",
        "properties": {
          "dtype": "number",
          "std": 0.6048128003332052,
          "min": 6.8,
          "max": 9.92,
          "num_unique_values": 184,
          "samples": [
            9.6, 8.9, 8.24
          ],
          "semantic_type": ""
        },
        "description": "",
        "column": "Research",
        "properties": {
          "dtype": "number",
          "std": 0,
          "min": 0,
          "max": 1,
          "num_unique_values": 2,
          "samples": [
            0, 1
          ],
          "semantic_type": ""
        },
        "description": ""
      }
    ]
  }
}
```

```

{"column": "Chance of Admit ", "properties": {"dtype": "number", "std": 0.1411404039503023, "min": 0.34, "max": 0.97, "num_unique_values": 61, "samples": [0.92, 0.9], "semantic_type": "", "description": ""}
}, {"type": "dataframe", "variable_name": "df"}

```

shape of the data

```
df.shape
```

```
(500, 9)
```

to observe the data type

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 500 entries, 0 to 499
```

```
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	Serial No.	500 non-null	int64
1	GRE Score	500 non-null	int64
2	TOEFL Score	500 non-null	int64
3	University Rating	500 non-null	int64
4	SOP	500 non-null	float64
5	LOR	500 non-null	float64
6	CGPA	500 non-null	float64
7	Research	500 non-null	int64
8	Chance of Admit	500 non-null	float64

```
dtypes: float64(4), int64(5)
```

```
memory usage: 35.3 KB
```

```
df = df.dropna()
```

```
cat_cols = ['University Rating', 'SOP', 'LOR', 'Research']
```

```
num_cols = ['GRE Score', 'TOEFL Score', 'CGPA']
```

```
target = 'Chance of Admit'
```

to know the name of the columns

```
df.columns
```

```

Index(['Serial No.', 'GRE Score', 'TOEFL Score', 'University Rating',
      'SOP',
      'LOR ', 'CGPA', 'Research', 'Chance of Admit '],
      dtype='object')

```

```
df.columns = df.columns.str.strip()
```

```
df.columns
```

```
Index(['Serial No.', 'GRE Score', 'TOEFL Score', 'University Rating',
      'SOP',
      'LOR', 'CGPA', 'Research', 'Chance of Admit'],
      dtype='object')
```

Missing or Null Values in data

```
df.isnull().sum()
```

```
Serial No.      0
GRE Score      0
TOEFL Score     0
University Rating 0
SOP            0
LOR            0
CGPA           0
Research        0
Chance of Admit 0
dtype: int64
```

The dataset looks good as we can observe there is no missing or null data

```
df.nunique()
```

```
Serial No.      500
GRE Score       49
TOEFL Score     29
University Rating 5
SOP            9
LOR            9
CGPA          184
Research        2
Chance of Admit 61
dtype: int64
```

```
df['Serial No.'].nunique()
```

```
500
```

lets drop the serial number as it's not a feature required for prediction

```
df.drop('Serial No.', axis=1, inplace=True)
```

```
df
```

```
{"summary":{"\n  \"name\": \"df\",\n  \"rows\": 500,\n  \"fields\": [\n    {\n      \"column\": \"GRE Score\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 11,\n        \"min\": 290,\n        \"max\": 340,\n        \"num_unique_values\": 49,\n        \"samples\": [\n          307,\n          335,\n          297\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"TOEFL Score\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
```

```

6,\n          \"min\": 92,\n          \"max\": 120,\n          \"num_unique_values\": 29,\n          \"samples\": [\n          94,\n          119,\n          112\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          }\n          },\n          {\n          \"column\":\n          \"University Rating\",\n          \"properties\": {\n          \"dtype\":\n          \"number\",\n          \"std\": 1,\n          \"min\": 1,\n          \"max\": 5,\n          \"num_unique_values\": 5,\n          \"samples\":\n          [\n          3,\n          1,\n          2\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          }\n          },\n          {\n          \"column\": \"SOP\",\n          \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.9910036207566069,\n          \"min\": 1.0,\n          \"max\": 5.0,\n          \"num_unique_values\":\n          9,\n          \"samples\": [\n          1.0,\n          4.0,\n          5.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          }\n          },\n          {\n          \"column\":\n          \"LOR\",\n          \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.9254495738978181,\n          \"min\": 1.0,\n          \"max\":\n          5.0,\n          \"num_unique_values\": 9,\n          \"samples\": [\n          5.0,\n          3.5,\n          1.5\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          }\n          },\n          {\n          \"column\": \"CGPA\",\n          \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.6048128003332052,\n          \"min\": 6.8,\n          \"max\": 9.92,\n          \"num_unique_values\":\n          184,\n          \"samples\": [\n          9.6,\n          8.9,\n          8.24\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          }\n          },\n          {\n          \"column\":\n          \"Research\",\n          \"properties\": {\n          \"dtype\":\n          \"number\",\n          \"std\": 0,\n          \"min\": 0,\n          \"max\": 1,\n          \"num_unique_values\": 2,\n          \"samples\":\n          [\n          0,\n          1\n          ],\n          \"semantic_type\":\n          \"\",\n          \"description\": \"\",\n          }\n          },\n          {\n          \"column\": \"Chance of Admit\",\n          \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.1411404039503023,\n          \"min\": 0.34,\n          \"max\": 0.97,\n          \"num_unique_values\":\n          61,\n          \"samples\": [\n          0.92,\n          0.9\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          }\n          }\n          ]\n          }],\n          \"type\": \"dataframe\", \"variable_name\": \"df\"}

```

```
df.describe()
```

```

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"GRE Score\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 134.31959598717793,\n        \"min\": 11.295148372354694,\n        \"max\": 500.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          317.0,\n          500.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"TOEFL Score\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\":\n

```

```

148.54698537663884,\n          \n\"min\": 6.081867659564522,\n
\n\"max\": 500.0,\n          \n\"num_unique_values\": 8,\n
\n\"samples\": [\n          107.192,\n          107.0,\n          500.0\n
\n          ],\n          \n\"semantic_type\": \"\",\n
\n\"description\": \"\"\n          }\n          },\n          {\n          \n\"column\":
\n\"University Rating\",\n          \n\"properties\": {\n          \n\"dtype\":
\n\"number\",\n          \n\"std\": 175.8093363236959,\n          \n\"min\":
1.0,\n          \n\"max\": 500.0,\n          \n\"num_unique_values\": 8,\n
\n\"samples\": [\n          3.114,\n          3.0,\n          500.0\n
\n          ],\n          \n\"semantic_type\": \"\",\n          \n\"description\": \"\"\n
\n          },\n          {\n          \n\"column\": \"SOP\",\n          \n\"properties\": {\n
\n          \n\"dtype\": \"number\",\n          \n\"std\": 175.75364204315028,\n
\n          \n\"min\": 0.9910036207566069,\n          \n\"max\": 500.0,\n
\n          \n\"num_unique_values\": 8,\n          \n\"samples\": [\n          3.374,\n
3.5,\n          500.0\n          ],\n          \n\"semantic_type\": \"\",\n
\n          \n\"description\": \"\"\n          }\n          },\n          {\n          \n\"column\":
\n\"LOR\",\n          \n\"properties\": {\n          \n\"dtype\": \"number\",\n
\n          \n\"std\": 175.72621272918164,\n          \n\"min\": 0.9254495738978181,\n
\n          \n\"max\": 500.0,\n          \n\"num_unique_values\": 8,\n
\n          \n\"samples\": [\n          3.484,\n          3.5,\n          500.0\n
\n          ],\n          \n\"semantic_type\": \"\",\n          \n\"description\": \"\"\n
\n          },\n          {\n          \n\"column\": \"CGPA\",\n          \n\"properties\":
{\n          \n\"dtype\": \"number\",\n          \n\"std\":
174.19317432229437,\n          \n\"min\": 0.6048128003332052,\n
\n          \n\"max\": 500.0,\n          \n\"num_unique_values\": 8,\n
\n          \n\"samples\": [\n          8.576439999999998,\n          8.56,\n
500.0\n          ],\n          \n\"semantic_type\": \"\",\n
\n          \n\"description\": \"\"\n          }\n          },\n          {\n          \n\"column\":
\n\"Research\",\n          \n\"properties\": {\n          \n\"dtype\":
\n\"number\",\n          \n\"std\": 176.57228090801308,\n          \n\"min\":
0.0,\n          \n\"max\": 500.0,\n          \n\"num_unique_values\": 5,\n
\n          \n\"samples\": [\n          0.56,\n          1.0,\n
0.49688407860903494\n          ],\n          \n\"semantic_type\": \"\",\n
\n          \n\"description\": \"\"\n          }\n          },\n          {\n          \n\"column\":
\n\"Chance of Admit\",\n          \n\"properties\": {\n          \n\"dtype\":
\n\"number\",\n          \n\"std\": 176.55754521339983,\n          \n\"min\":
0.1411404039503023,\n          \n\"max\": 500.0,\n
\n          \n\"num_unique_values\": 8,\n          \n\"samples\": [\n          0.72174,\n
0.72,\n          500.0\n          ],\n          \n\"semantic_type\": \"\",\n
\n          \n\"description\": \"\"\n          }\n          }\n          ]\n          }\", \"type\": \"dataframe\"}

```

Characteristics of Quantitative Data:

1. Measures of Central Tendency These indicate the center or "typical" value of a dataset:

Mean: The arithmetic average; sensitive to extreme values (outliers).

Median: The middle value when data is sorted; resistant to outliers and skewed data.

Mode: The most frequently occurring value; useful for understanding frequency

2. Measures of Spread (Dispersion) These quantify how much the data varies from the central tendency:

Range: Difference between the maximum and minimum values.

Variance: The average of squared deviations from the mean; shows how data is spread out.

Standard Deviation: Square root of variance; interpretable in the same units as the data.

Interquartile Range (IQR): The range between the 25th and 75th percentile; helps detect outliers.

3. Shape of the Distribution Describes the symmetry and peakedness of the data's distribution.

□ Skewness (Symmetry of Distribution) Indicates whether the distribution leans to the left or right:

Skewness $\approx 0 \rightarrow$ Data is fairly symmetrical.

Skewness between -1 and -0.5 or 0.5 and 1 $\rightarrow$ Moderately skewed.

Skewness < -1 or $> 1 \rightarrow$ Highly skewed.

Negative skew (left-skewed): Tail is on the left; mean $<$ median.

Positive skew (right-skewed): Tail is on the right; mean $>$ median.

Kurtosis (Peakedness of Distribution)

Describes the height and sharpness of the peak and tails:

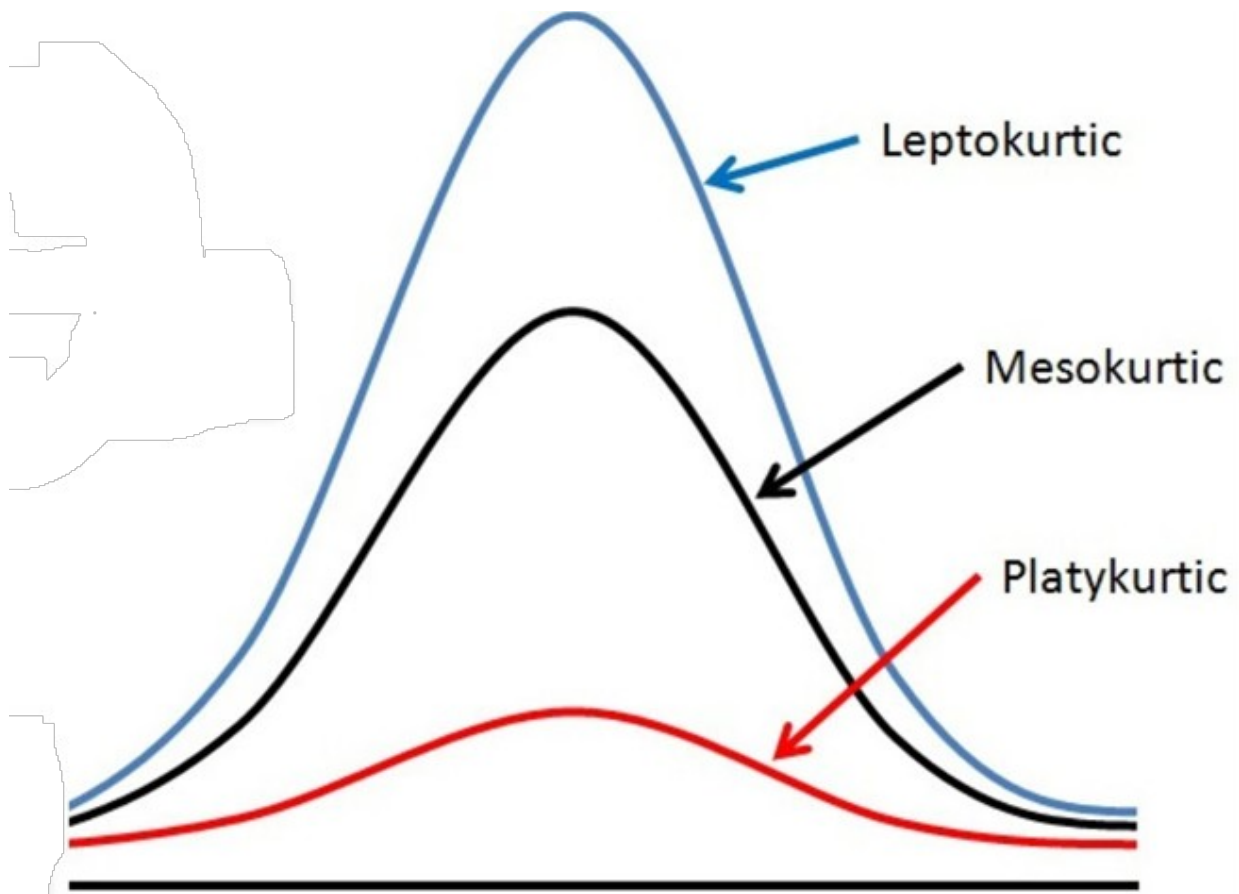
Leptokurtic (Kurtosis > 3): Tall, sharp peak with heavy tails; more prone to outliers.

Platykurtic (Kurtosis < 3): Flat distribution with light tails; fewer outliers.

Mesokurtic (Kurtosis $= 3$): Similar to normal distribution.

Excess Kurtosis = Kurtosis - 3

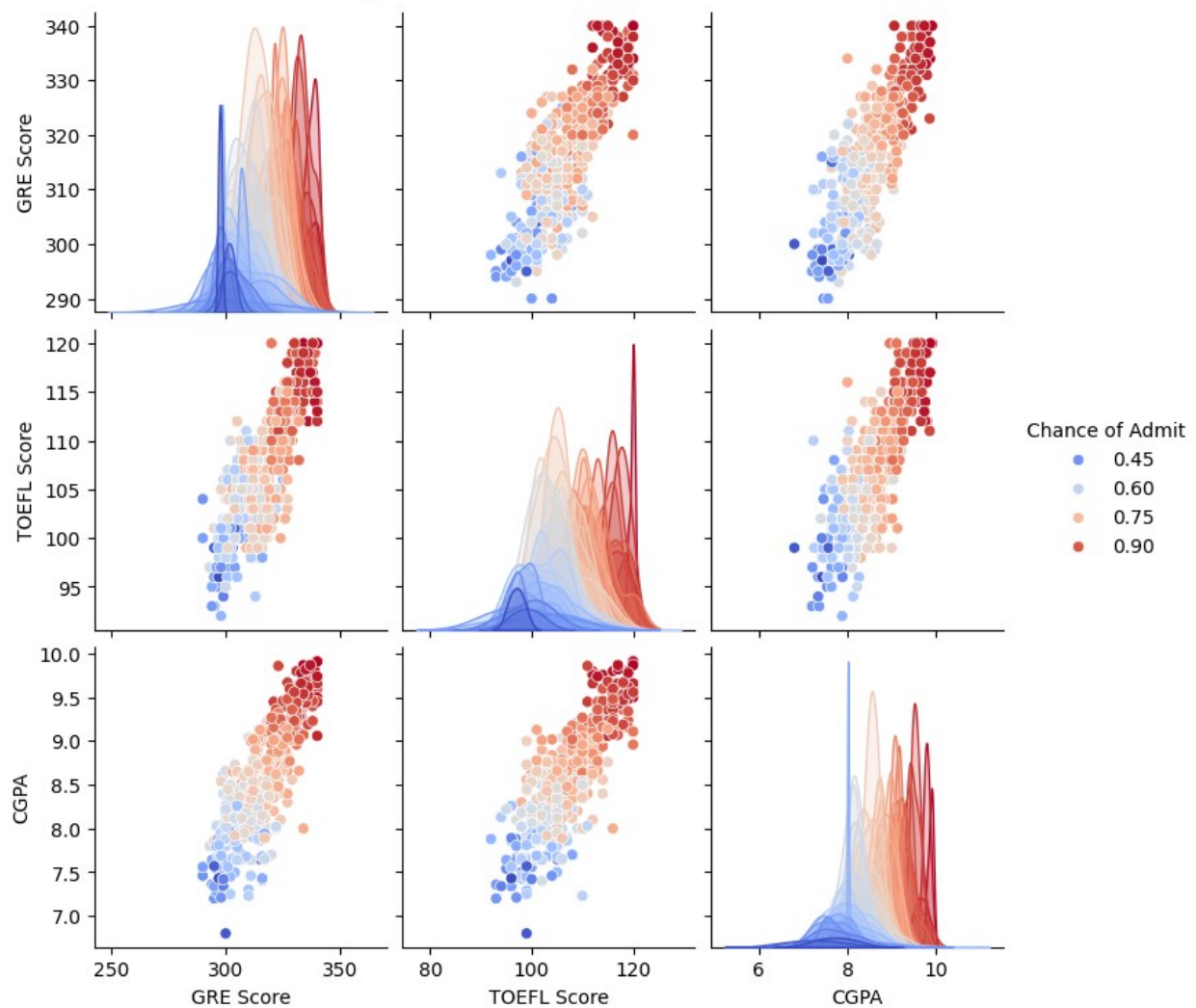
Makes normal distribution a baseline with kurtosis = 0



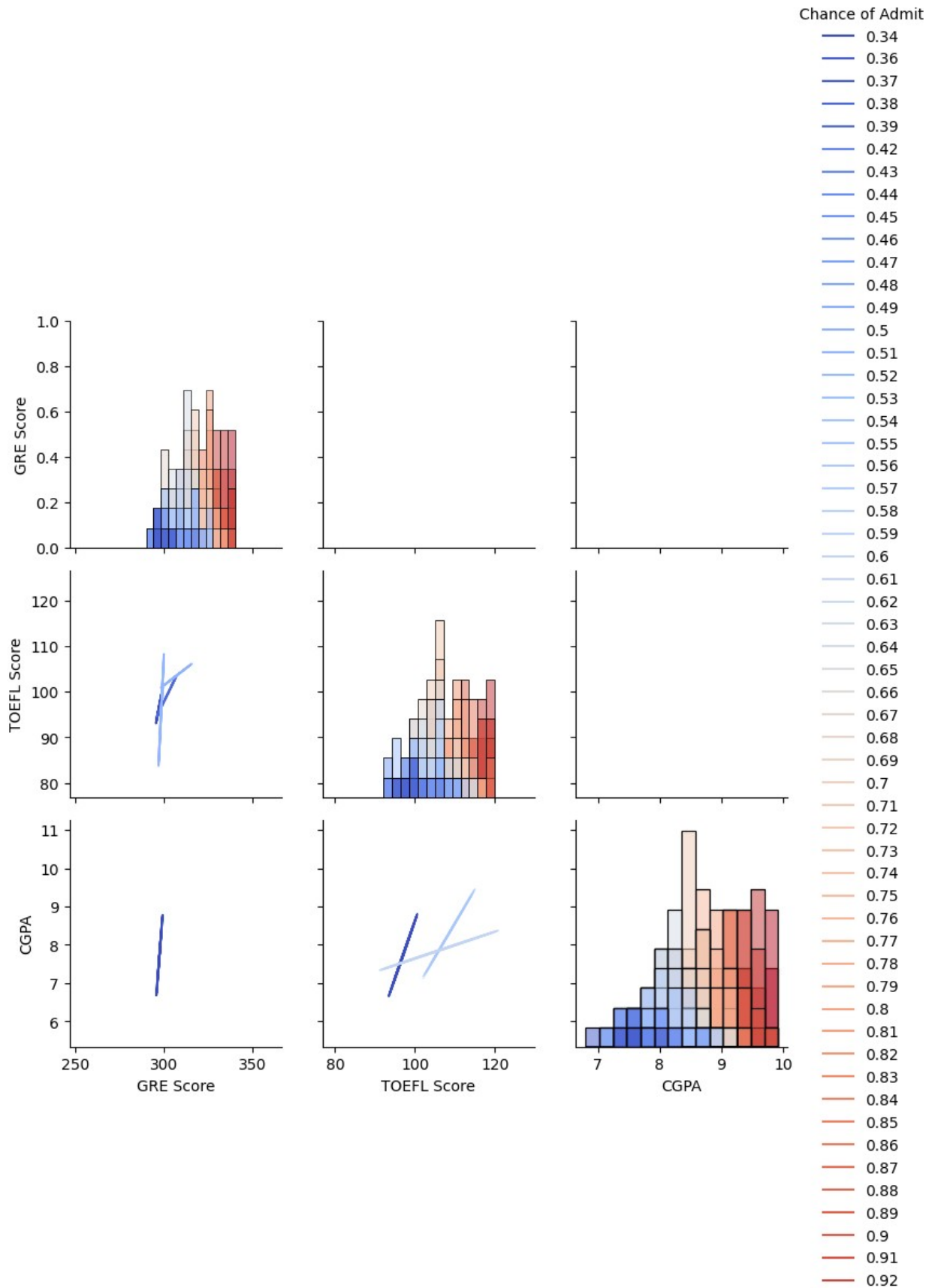
Multivariate Analysis

```
num_cols = ['GRE Score', 'TOEFL Score', 'CGPA', 'Chance of Admit']  
  
# Pairplot for multivariate analysis (target is 'Chance of Admit')  
sns.pairplot(df[num_cols], kind='scatter', hue='Chance of Admit',  
palette="coolwarm")  
  
# Add title  
plt.suptitle("Multivariate Analysis: Relationships Between Scores and  
Chance of Admit", y=1.02)  
  
plt.show()
```


Multivariate Analysis: Relationships Between Scores and Chance of Admit



```
# PairGrid for more customization (optional)
g = sns.PairGrid(df[num_cols], hue="Chance of Admit",
palette="coolwarm")
g.map_lower(sns.kdeplot)
g.map_diag(sns.histplot, kde_kws={'color': 'blue'})
g.add_legend()
plt.show()
```



BiVariate Analysis

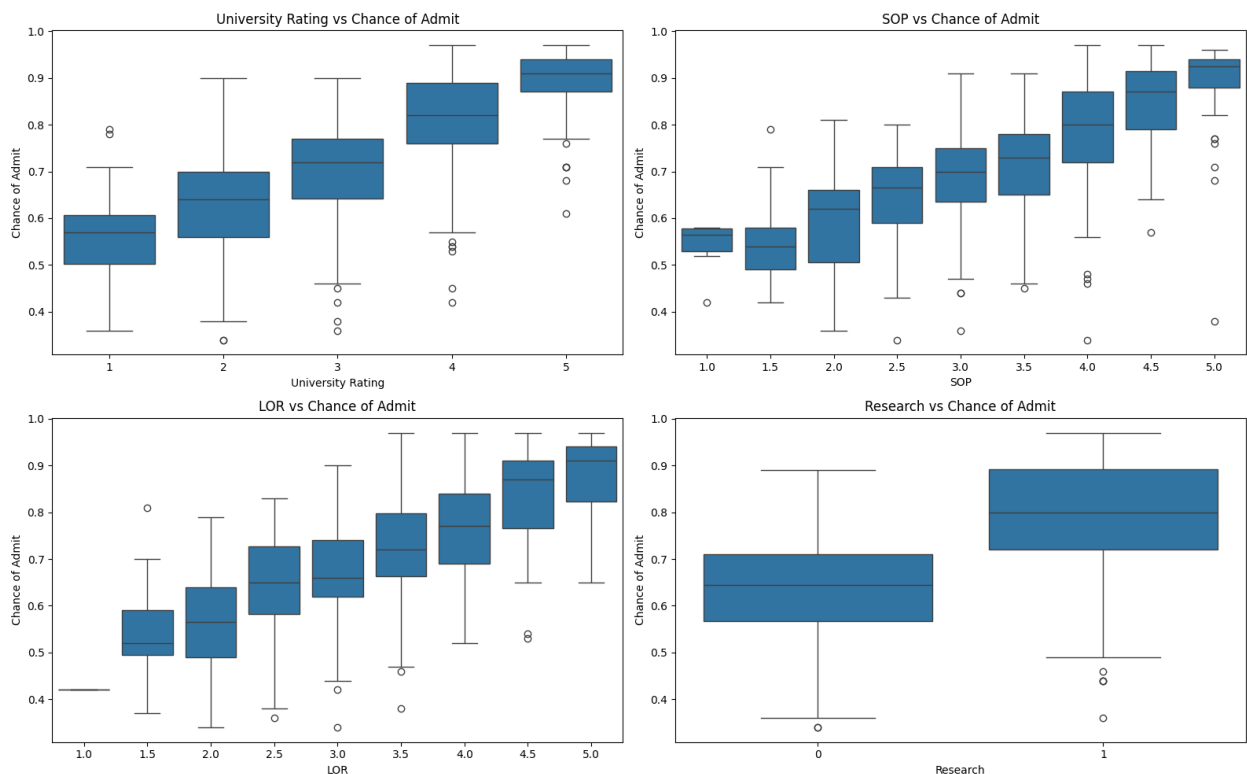
Categorical variables

1. 'University rating' vs 'Chance of Admit'
2. 'SOP' vs 'Chance of Admit'
3. 'LOR' vs 'Chance of Admit'
4. 'Research' vs 'Chance of Admit'

```
fig, axs = plt.subplots(2, 2, figsize=(16, 10))
axs = axs.flatten()

for i, cat in enumerate(cat_cols):
    sns.boxplot(x=df[cat], y=df[target], ax=axs[i])
    axs[i].set_title(f'{cat} vs {target}')
    axs[i].set_xlabel(cat)
    axs[i].set_ylabel('Chance of Admit')

plt.tight_layout()
plt.show()
```

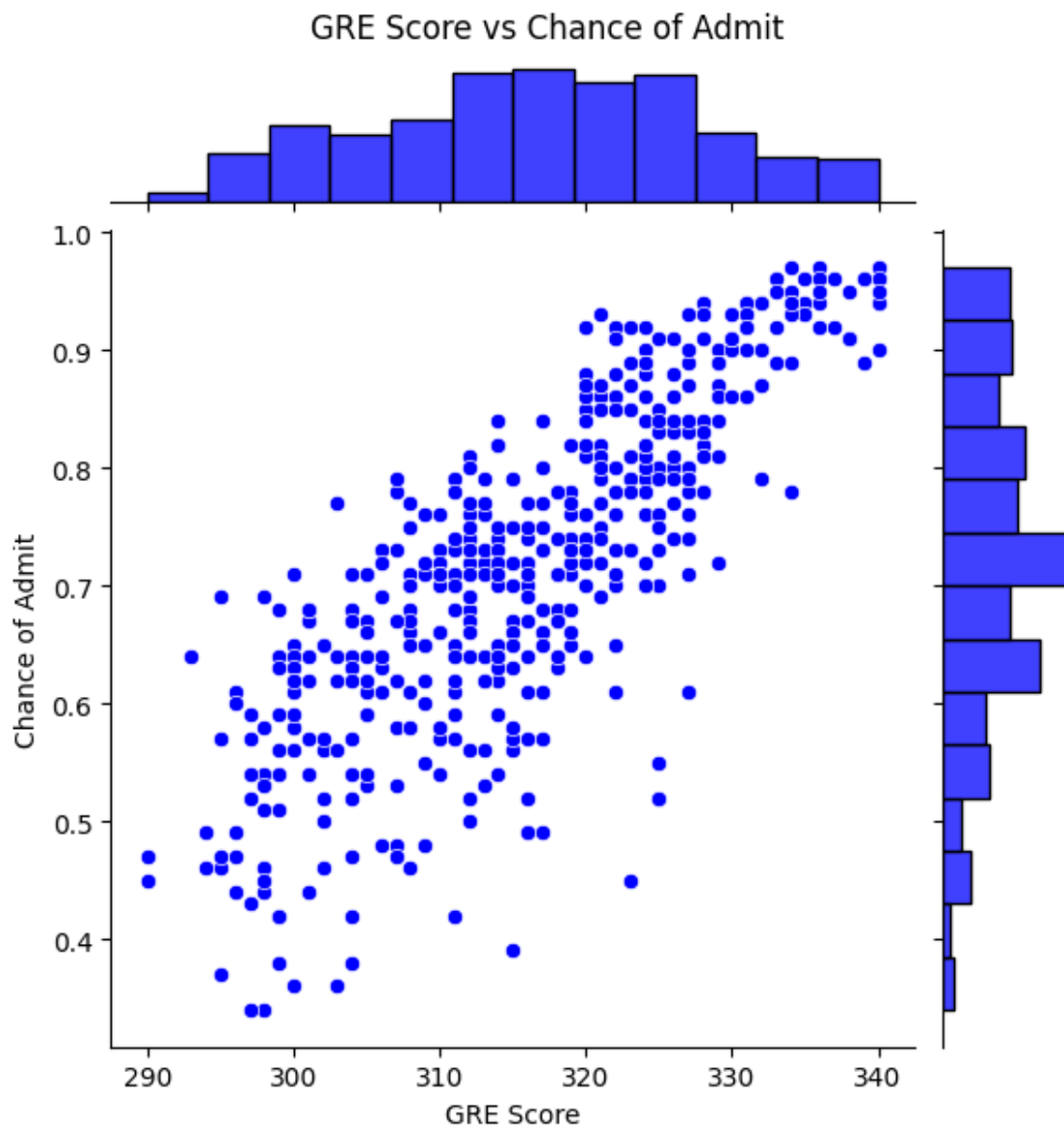


Numerical variables

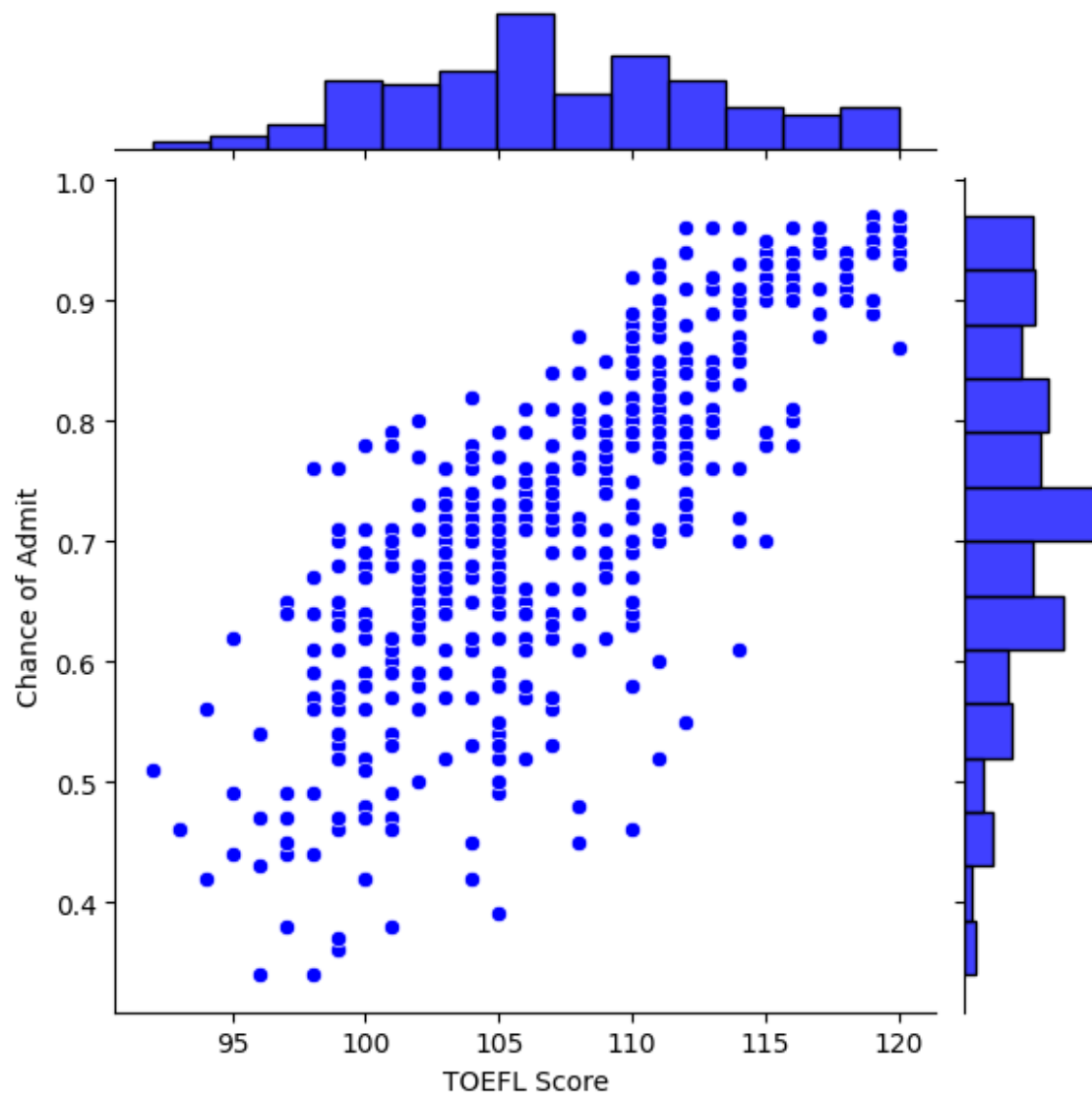
1. 'GRE Score' vs 'Chance of Admit'

2. 'TOEFL Score' vs 'Chance of Admit'
3. 'CGPA' vs 'Chance of Admit'

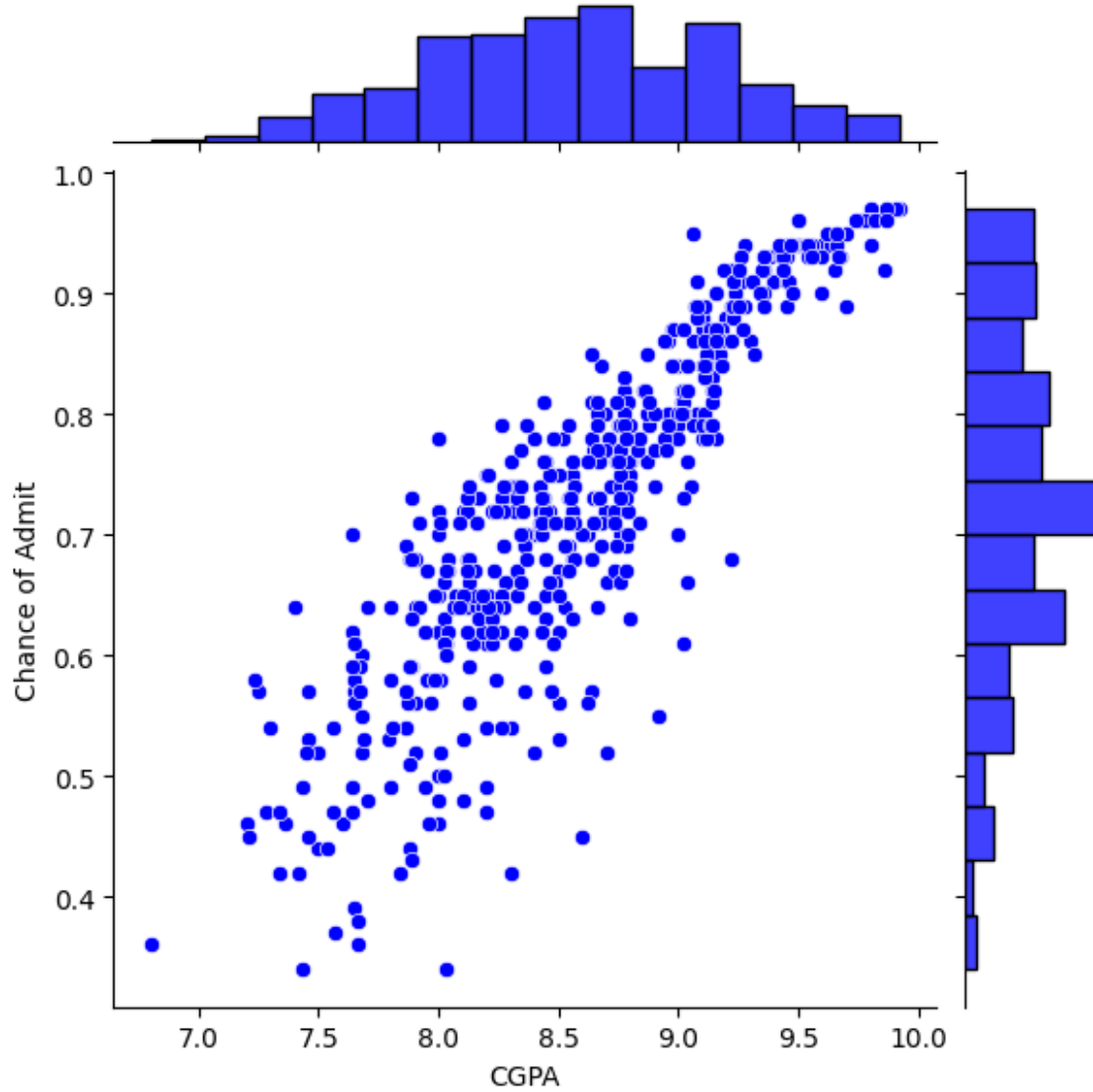
```
for col in num_cols:  
    sns.jointplot(x=df[col], y=df[target], kind='scatter',  
color='blue')  
    plt.suptitle(f'{col} vs {target}', y=1.02)  
    plt.show()
```

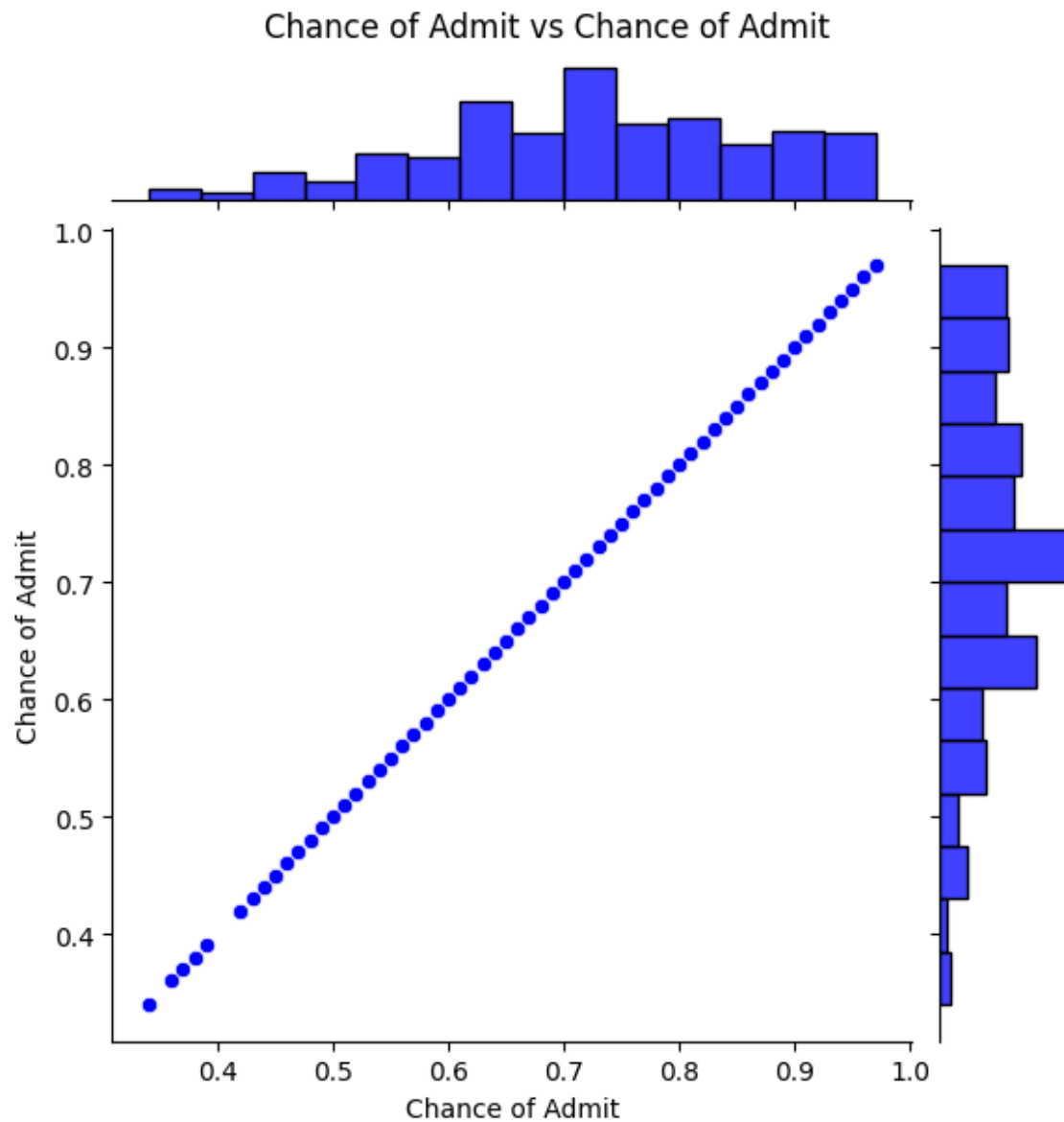


TOEFL Score vs Chance of Admit

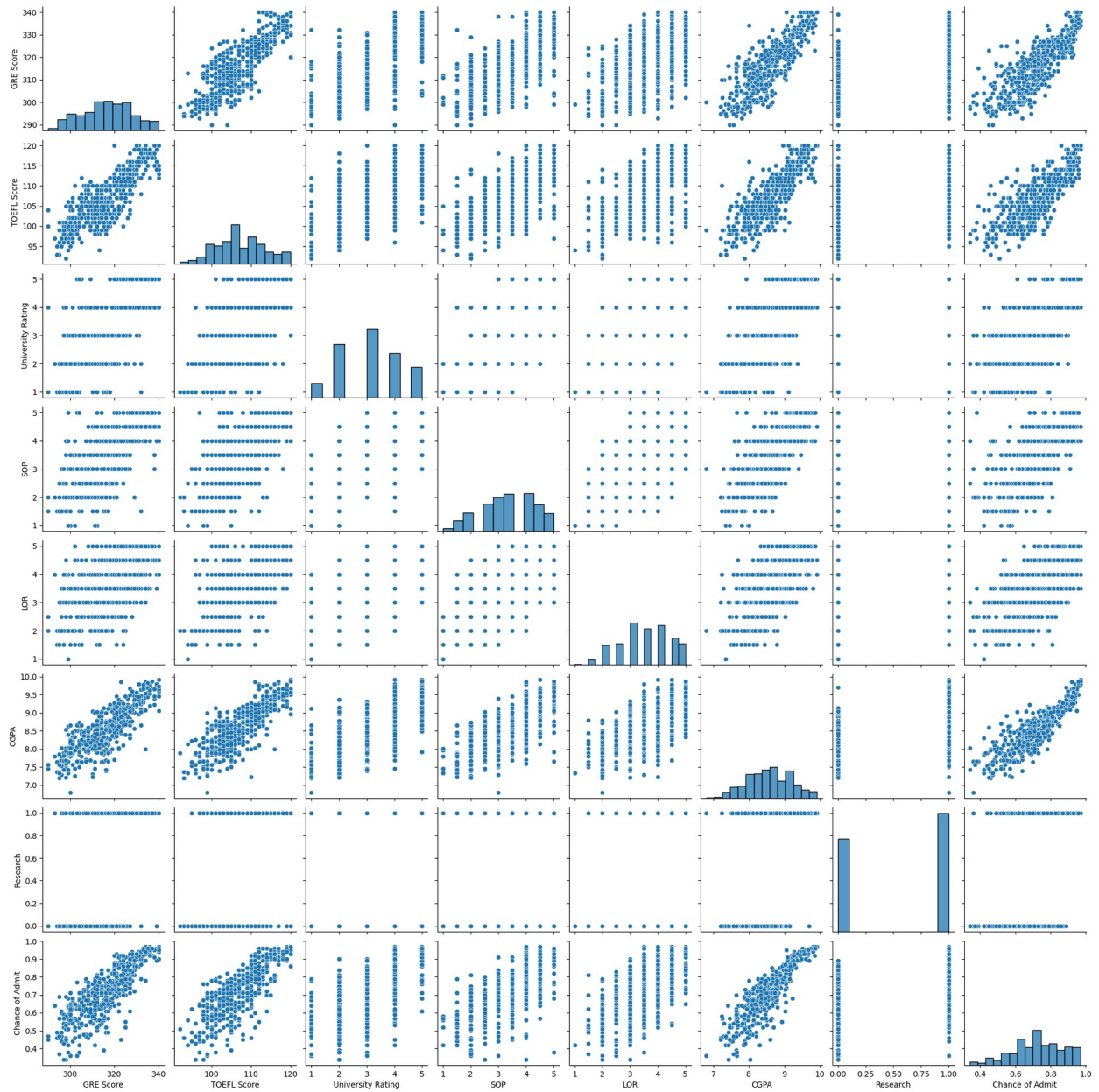


CGPA vs Chance of Admit





```
sns.pairplot(df)  
<seaborn.axisgrid.PairGrid at 0x7f7da68be010>
```



While university ranking, rating of SOP and LOR are also have impact on chances of admit, research is the only variable which does not have much of an impact

We can observe from the scatterplot that, the values of university ranking, SOP, LOR and research are not continuous. We can convert these columns to categorical variables.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
#   Column              Non-Null Count  Dtype
---  -

```


0	GRE Score	500	non-null	int64
1	TOEFL Score	500	non-null	int64
2	University Rating	500	non-null	int64
3	SOP	500	non-null	float64
4	LOR	500	non-null	float64
5	CGPA	500	non-null	float64
6	Research	500	non-null	int64
7	Chance of Admit	500	non-null	float64

dtypes: float64(4), int64(4)

memory usage: 31.4 KB

df.describe()

```
{
  "summary": {
    "name": "df",
    "rows": 8,
    "fields": [
      {
        "column": "GRE Score",
        "properties": {
          "dtype": "number",
          "std": 134.31959598717793,
          "min": 11.295148372354694,
          "max": 500.0,
          "num_unique_values": 8,
          "samples": [
            316.472,
            317.0,
            500.0
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "TOEFL Score",
        "properties": {
          "dtype": "number",
          "std": 148.54698537663884,
          "min": 6.081867659564522,
          "max": 500.0,
          "num_unique_values": 8,
          "samples": [
            107.192,
            107.0,
            500.0
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "University Rating",
        "properties": {
          "dtype": "number",
          "std": 175.8093363236959,
          "min": 1.0,
          "max": 500.0,
          "num_unique_values": 8,
          "samples": [
            3.114,
            3.0,
            500.0
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "SOP",
        "properties": {
          "dtype": "number",
          "std": 175.75364204315028,
          "min": 0.9910036207566069,
          "max": 500.0,
          "num_unique_values": 8,
          "samples": [
            3.374,
            3.5,
            500.0
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "LOR",
        "properties": {
          "dtype": "number",
          "std": 175.72621272918164,
          "min": 0.9254495738978181,
          "max": 500.0,
          "num_unique_values": 8,
          "samples": [
            3.484,
            3.5,
            500.0
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "CGPA",
        "properties": {
          "dtype": "number",
          "std": 174.19317432229437,
          "min": 0.6048128003332052,
          "max": 500.0,
          "num_unique_values": 8,
          "samples": [
            8.576439999999998,
            8.56,
            500.0
          ],
          "semantic_type": "",
          "description": ""
        }
      }
    ]
  }
}
```

```

{"Research": "\n", "properties": {"dtype": "\n", "std": 176.57228090801308, "min": 0.0, "max": 500.0, "num_unique_values": 5, "samples": [0.56, 1.0, 0.49688407860903494]}, "semantic_type": "\n", "description": "\n"}, {"Chance of Admit": "\n", "properties": {"dtype": "\n", "std": 176.55754521339983, "min": 0.1411404039503023, "max": 500.0, "num_unique_values": 8, "samples": [0.72174, 0.72, 500.0]}, "semantic_type": "\n", "description": "\n"}], "type": "dataframe"}

```

Chances of admit is a probability measure, which is within 0 to 1 which is good (no outlier or misleading data)

Range of GRE score looks between 290 to 340

Range of TOEFL score is between 92 to 120

CGPA rating between 6.8 to 9.92

```

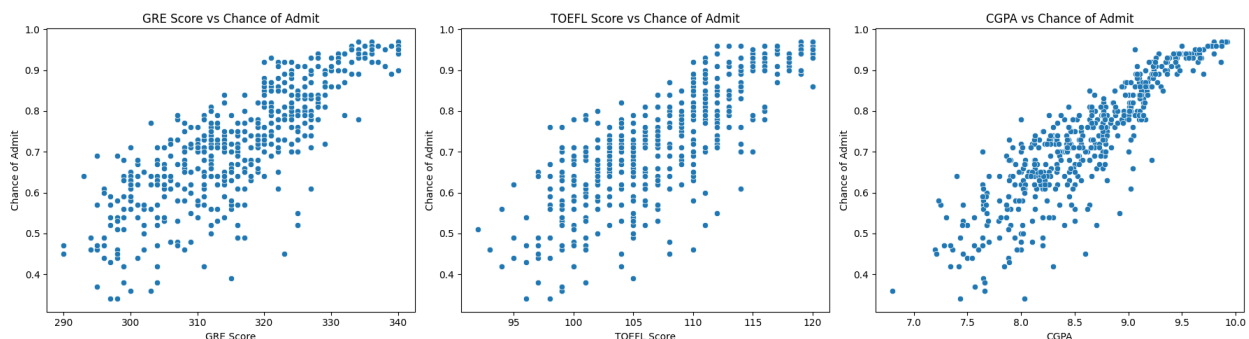
# df.describe(include = ['object', 'category'])

fig, axs = plt.subplots(1, 3, figsize=(18, 5))

for i in range(3):
    sns.scatterplot(x=num_cols[i], y=target, data=df, ax=axs[i])
    axs[i].set_title(f'{num_cols[i]} vs {target}')

plt.tight_layout()
plt.show()

```



Seems like there is a linear correlation between the continuous variables and the target variable.

```

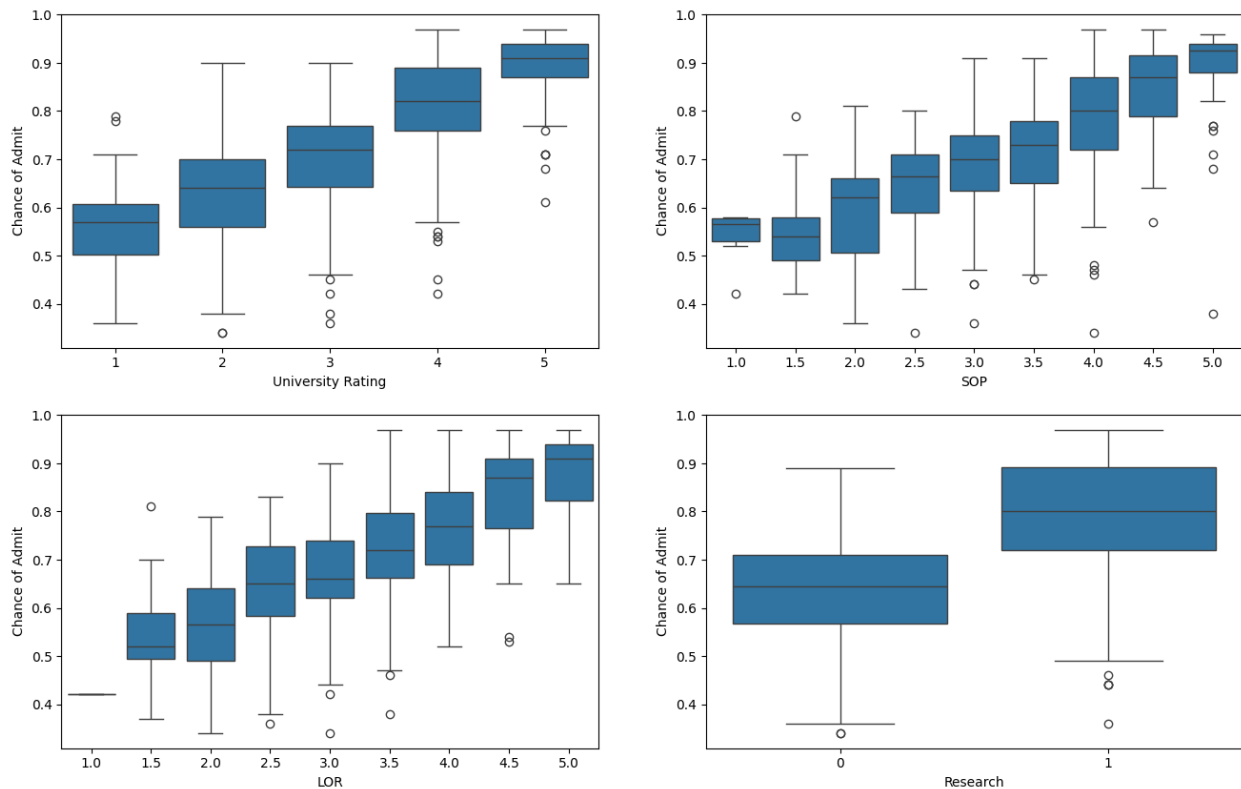
rows, cols = 2, 2
fig, axs = plt.subplots(rows, cols, figsize=(16, 10))

```

```

index = 0
for row in range(rows):
    for col in range(cols):
        sns.boxplot(x=cat_cols[index], y=target, data=df,
ax=axis[row,col])
        index += 1

```



As seen in the graphs, an increase in university rating generally corresponds to a higher chance of admission. Additionally, students with research experience tend to have a higher chance of admission compared to those without research experience

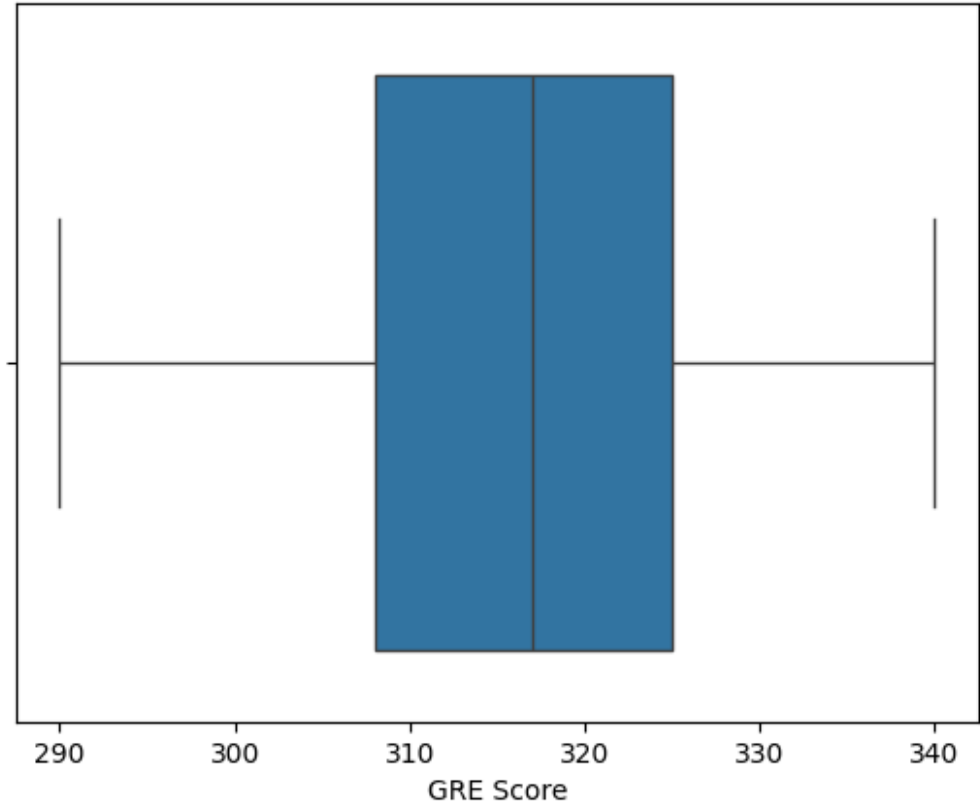
Univariate Analysis

```

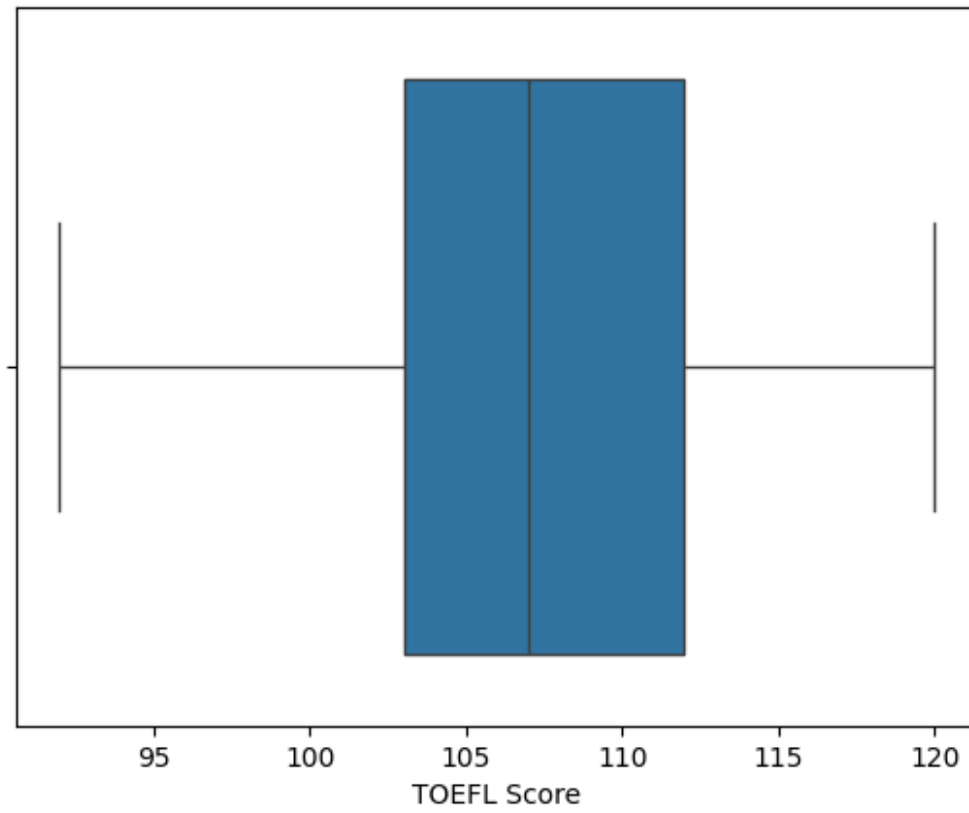
num_col = ['GRE Score', 'TOEFL Score', 'CGPA', 'Chance of Admit']
for col in num_col:
    sns.boxplot(x=df[col])
    plt.title(f'{col} distribution')
    plt.show()

```

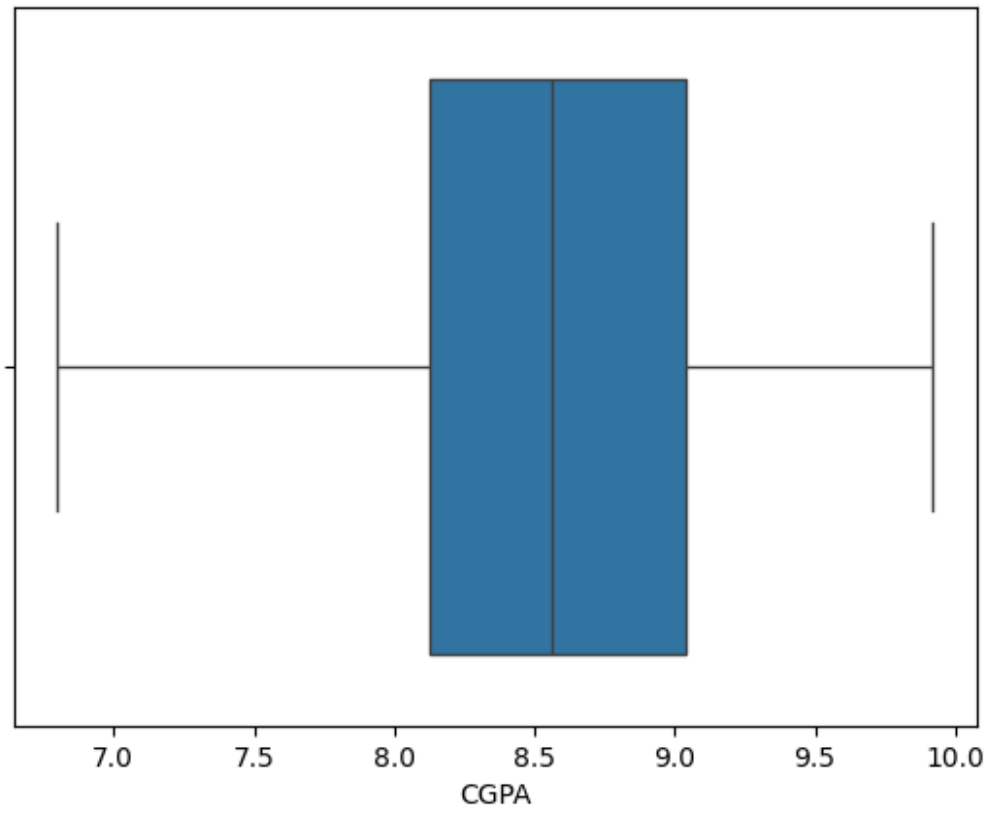
GRE Score distribution

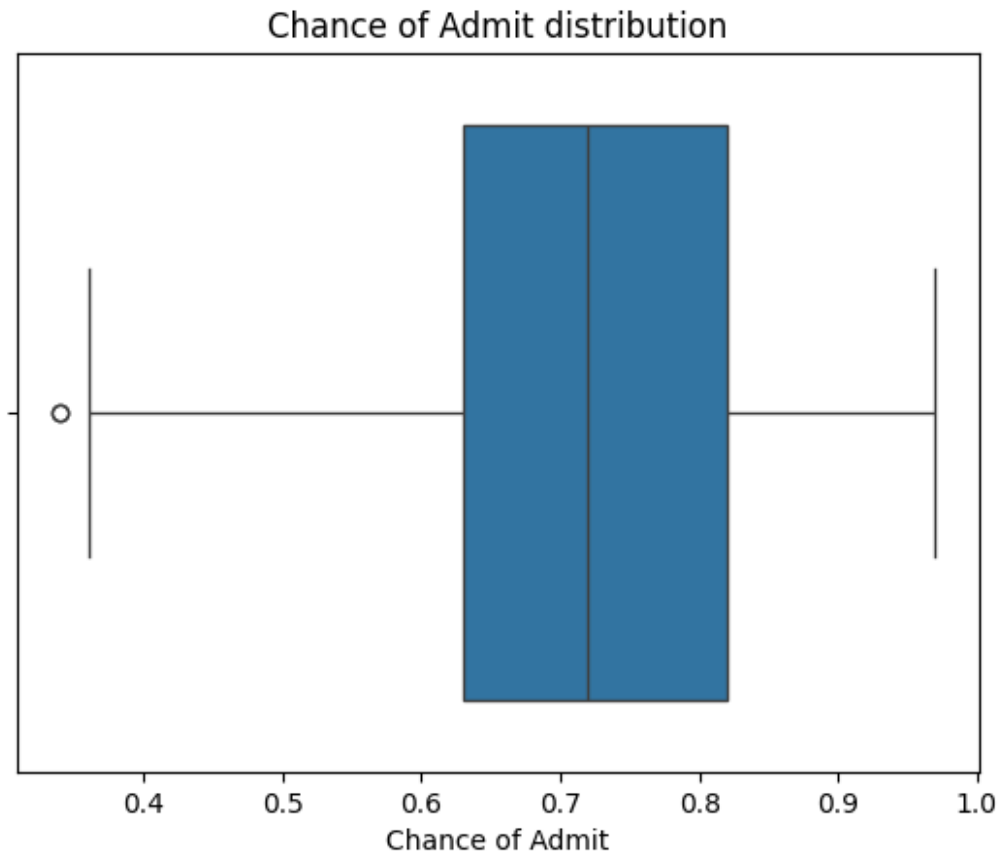


TOEFL Score distribution



CGPA distribution

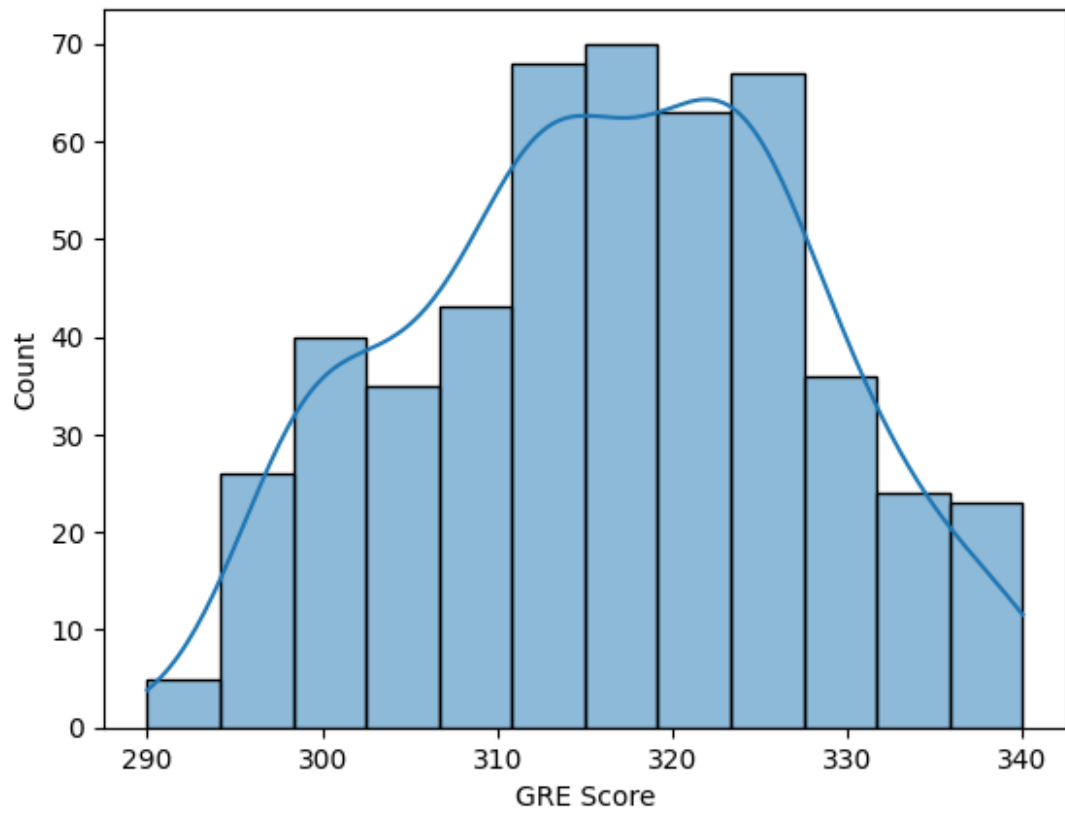




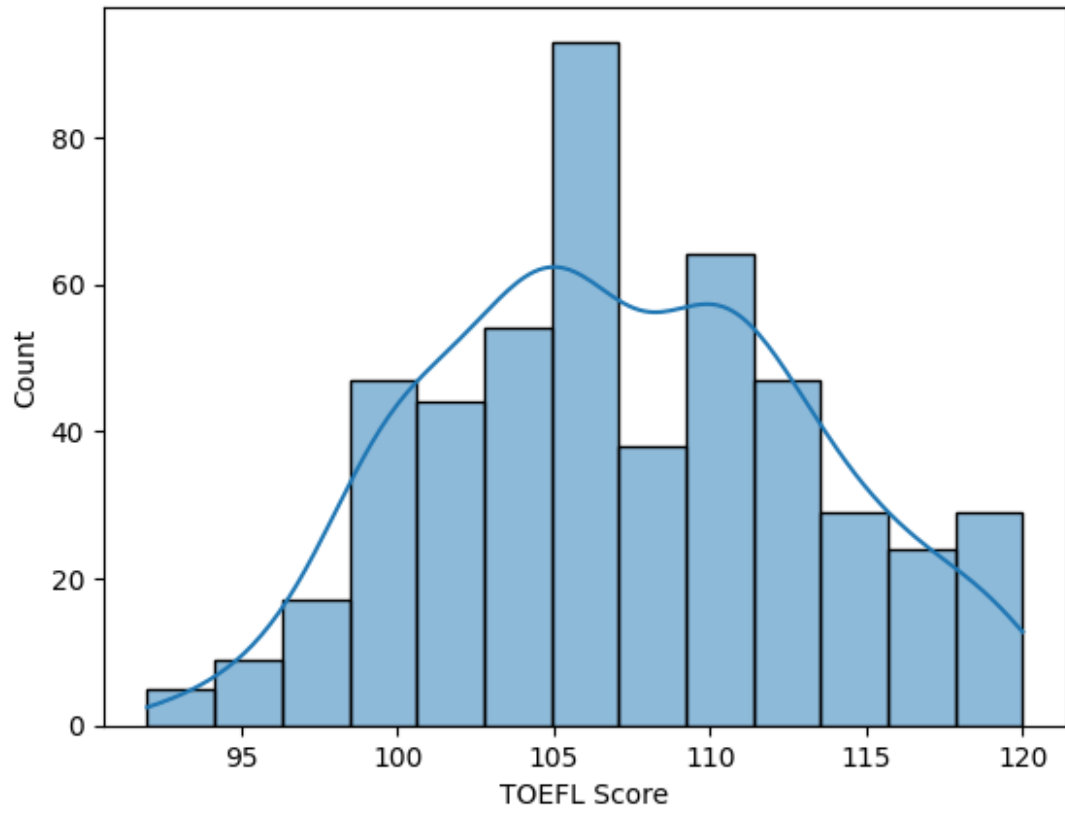
Based on the above graph, we do not have any outliers

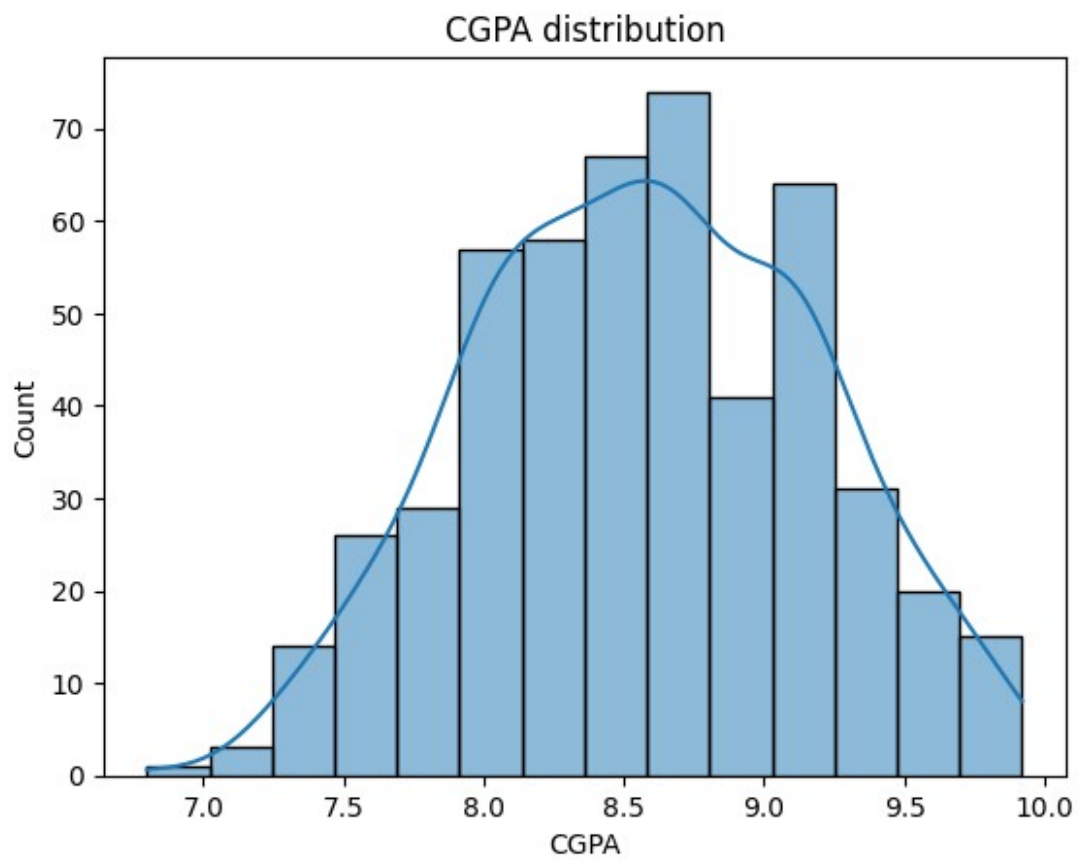
```
# GRE Score, TOEFL Score, CGPA and Chance of Admit using histplot
for col in num_col:
    sns.histplot(df[col], kde=True)
    plt.title(f'{col} distribution')
    plt.show()
```

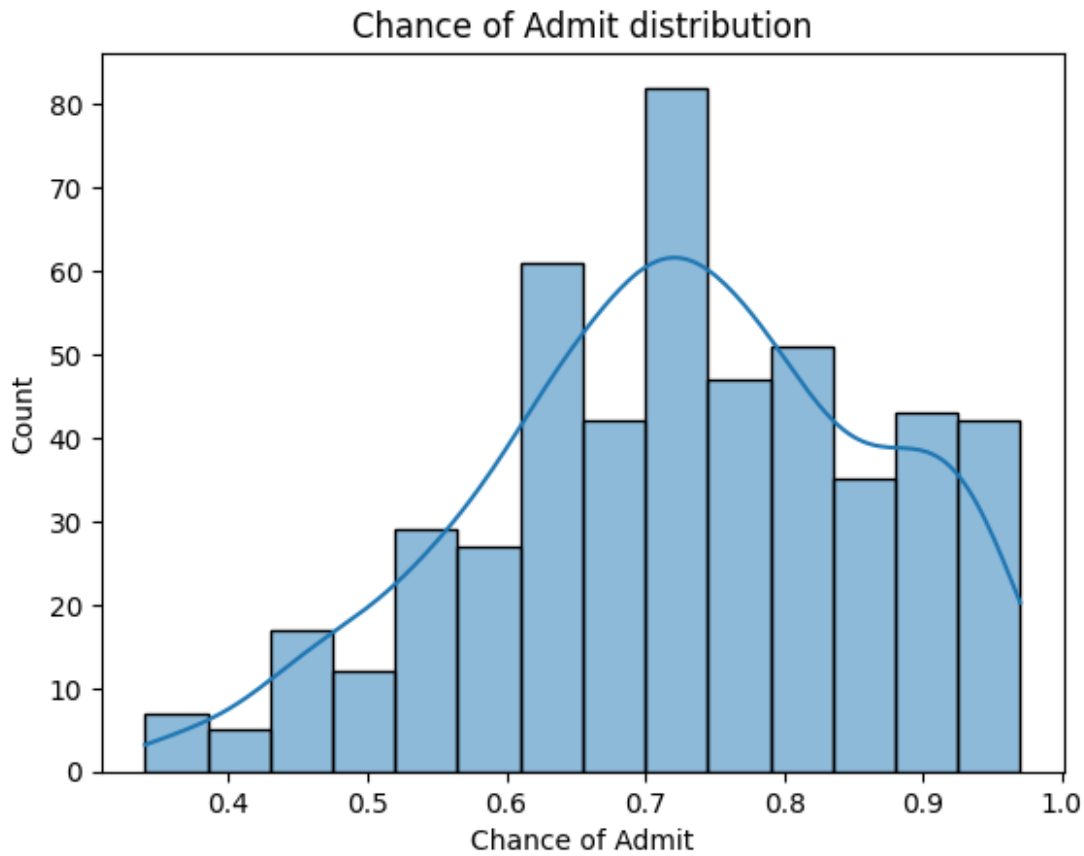
GRE Score distribution



TOEFL Score distribution







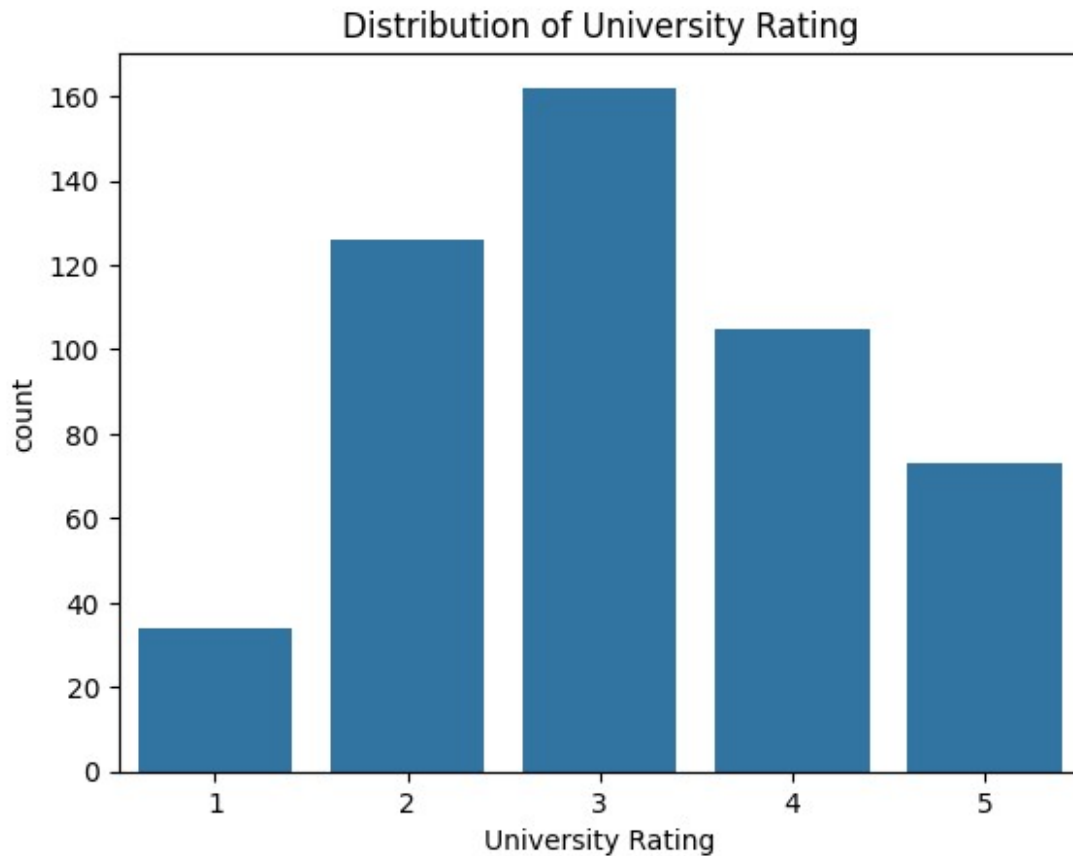
GRE scores are between 290 and 340, with maximum students scoring in the range 310-330

TOEFL scores are between 90 and 120, with maximum students scoring around 105

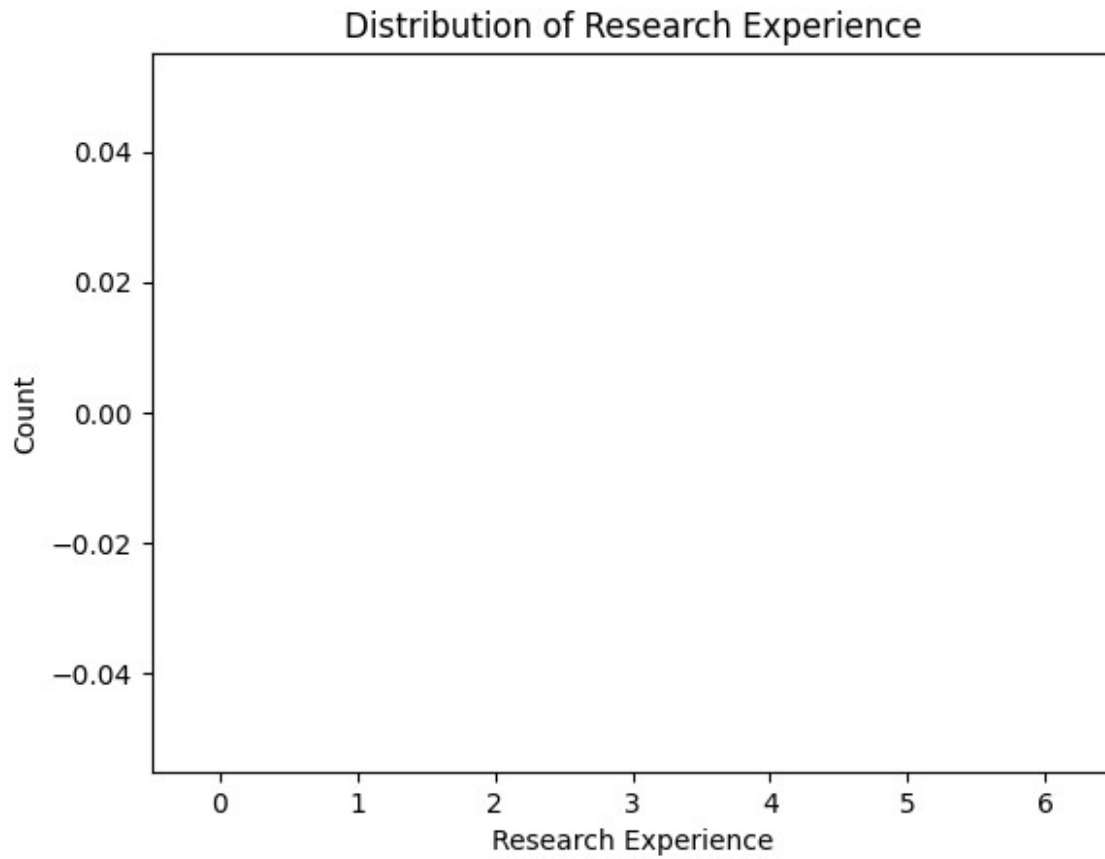
CGPA ranges between 7 and 10, with maximum students scoring around 8.5

Chance of Admit is a probability percentage between 0 and 1, with maximum students scoring around 70%-75%

```
sns.countplot(data=df, x='University Rating')  
plt.title(f'Distribution of University Rating')  
plt.show()
```

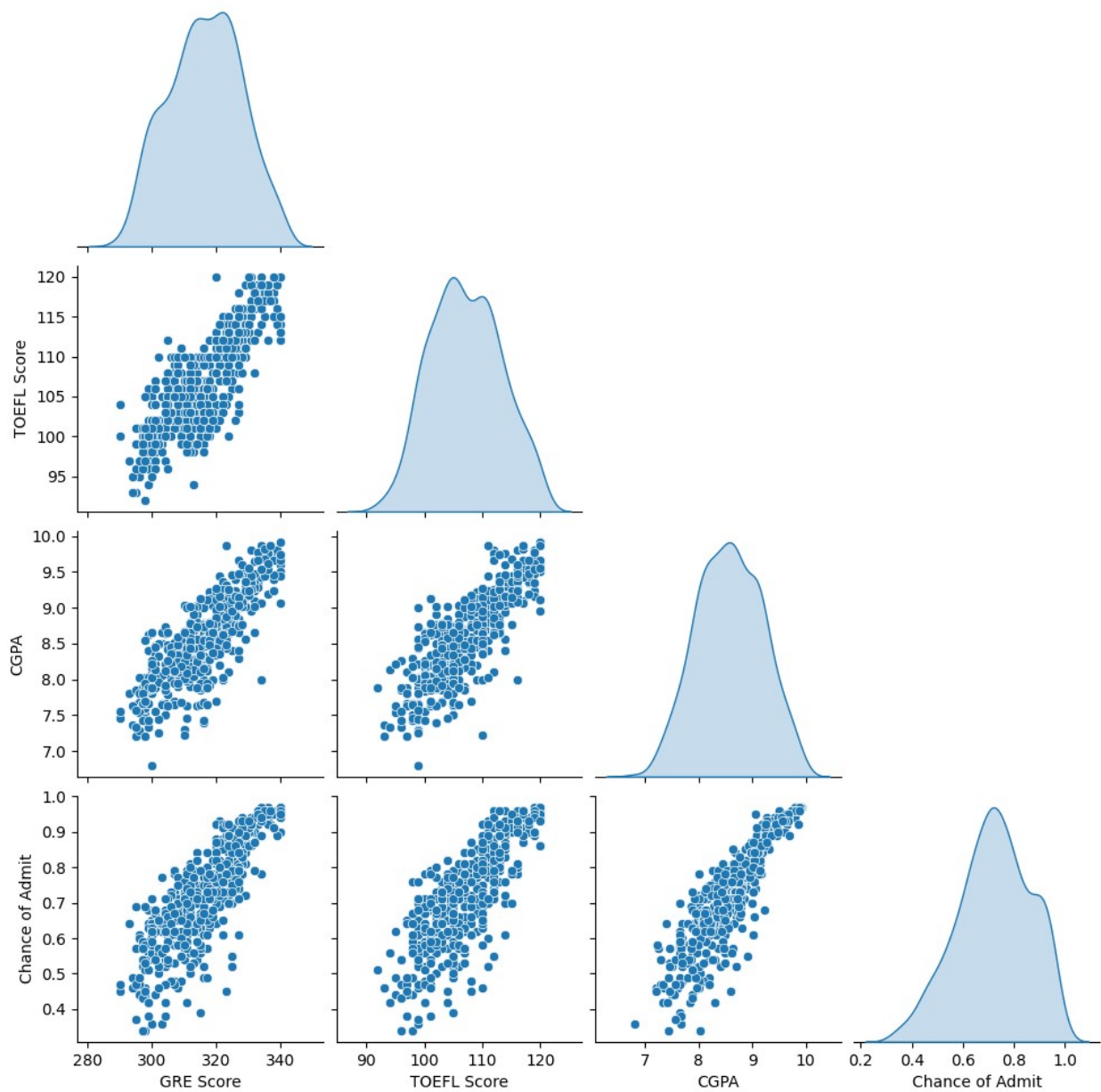


```
df['Research Label'] = df['Research'].map({False: 'No Experience',  
True: 'Experience'})  
sns.countplot(data=df, x='Research Label')  
plt.title('Distribution of Research Experience')  
plt.xlabel('Research Experience')  
plt.ylabel('Count')  
plt.show()
```

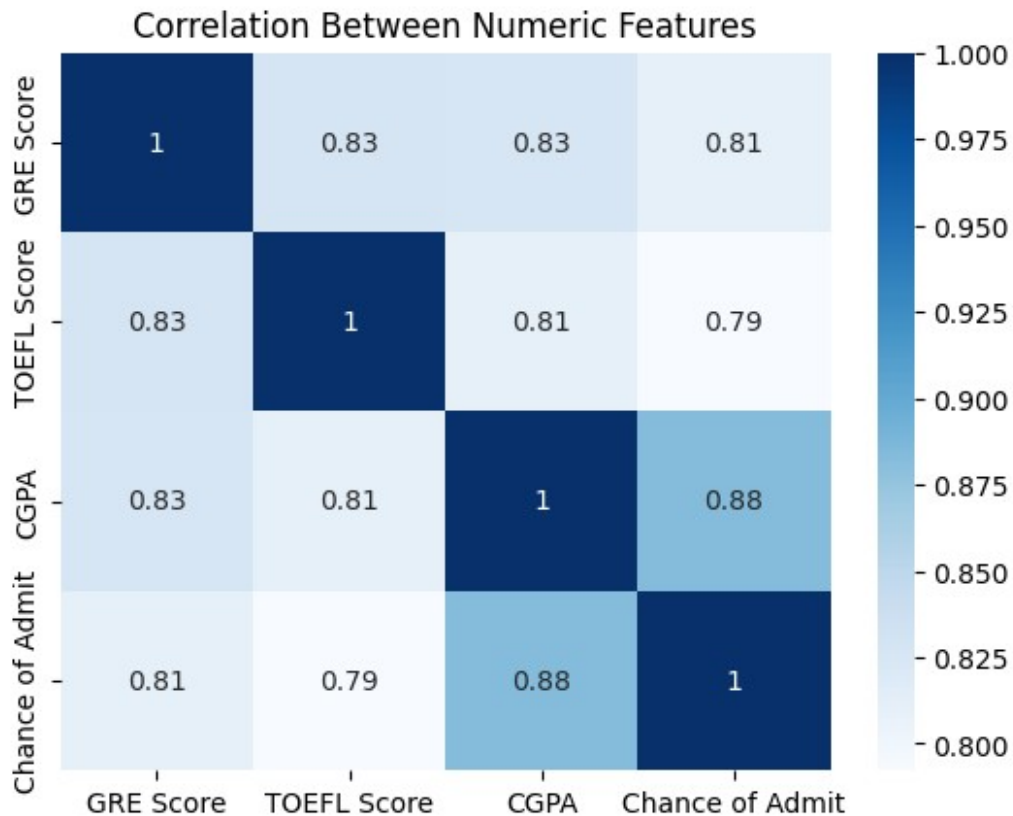


```
sns.pairplot(df[num_col], diag_kind='kde', corner=True)  
plt.suptitle("Pairplot of Exam Scores and Chance of Admit", y=1.02)  
plt.show()
```

Pairplot of Exam Scores and Chance of Admit



```
sns.heatmap(df[num_col].corr(), annot=True, cmap="Blues")  
  
plt.title('Correlation Between Numeric Features')  
plt.show()
```



Consistent with the insights from the pairplot, the correlation matrix confirms that exam scores — including CGPA, GRE, and TOEFL — have a strong positive correlation with the Chance of Admit. Interestingly, these scores are also highly correlated with one another, indicating that strong academic performance tends to be reflected across all exam metrics.

Data Processing

```
df.duplicated().sum()
np.int64(0)
df.head()

{"repr_error": "Out of range float values are not JSON compliant:
nan", "type": "dataframe", "variable_name": "df"}

df = df.drop(columns=['Research Label'])
X = df.drop(columns=[target])
y = df[target]
X.head()

{"summary": "{\\n  \\\"name\\\": \\\"X\\\",\\n  \\\"rows\\\": 500,\\n  \\\"fields\\\": [\\n
{\\n    \\\"column\\\": \\\"GRE Score\\\",\\n    \\\"properties\\\": {\\n
\\\"dtype\\\": \\\"number\\\",\\n    \\\"std\\\": 11,\\n    \\\"min\\\": 290,\\n
```

```

{"max": 340, "num_unique_values": 49, "samples": [307, 335, 297], "semantic_type": "", "description": ""}
{"column": "TOEFL Score", "dtype": "number", "std": 6, "min": 92, "max": 120, "num_unique_values": 29, "samples": [119, 112, 94], "semantic_type": "", "description": ""}
{"column": "University Rating", "dtype": "number", "std": 1, "min": 1, "max": 5, "num_unique_values": 5, "samples": [3, 1, 2], "semantic_type": "", "description": ""}
{"column": "SOP", "dtype": "number", "std": 0.9910036207566069, "min": 1.0, "max": 5.0, "num_unique_values": 9, "samples": [1.0, 4.0, 5.0], "semantic_type": "", "description": ""}
{"column": "LOR", "dtype": "number", "std": 0.9254495738978181, "min": 1.0, "max": 5.0, "num_unique_values": 9, "samples": [3.5, 1.5, 5.0], "semantic_type": "", "description": ""}
{"column": "CGPA", "dtype": "number", "std": 0.6048128003332052, "min": 6.8, "max": 9.92, "num_unique_values": 184, "samples": [9.6, 8.9, 8.24], "semantic_type": "", "description": ""}
{"column": "Research", "dtype": "number", "std": 0, "min": 0, "max": 1, "num_unique_values": 2, "samples": [0, 1], "semantic_type": "", "description": ""}
}, {"type": "dataframe", "variable_name": "X"}

```

y

0	0.92
1	0.76
2	0.72
3	0.80
4	0.65
...	
495	0.87
496	0.96
497	0.93
498	0.73


```

499      0.84
Name: Chance of Admit, Length: 500, dtype: float64

# standardize the dataset
sc = StandardScaler()
X = sc.fit_transform(X)

# Split the data into training and test data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=1)

print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)

(350, 7) (350,)
(150, 7) (150,)

```

MODEL BUILDING

```

lr = LinearRegression()
lr.fit(X_train, y_train)

LinearRegression()

#r2 score on train data
r2_score(y_train,lr.predict(X_train))

0.8209843725364347

#r2 score on test data
r2_score(y_test,lr.predict(X_test))

0.8157672116057979

def AdjustedR2score(R2,n,d):
    return 1-(((1-R2)*(n-1))/(n-d-1))

y_pred = lr.predict(X_test)
print("MSE:",mean_squared_error(y_test,y_pred))
print("RMSE:",np.sqrt(mean_squared_error(y_test,y_pred)))
print("MAE :",mean_absolute_error(y_test,y_pred) )
print("r2_score:",r2_score(y_test,y_pred))
print("Adjusted R2 score :",
AdjustedR2score(r2_score(y_test,y_pred),len(X),X.shape[1]))

MSE: 0.0041259342367078004
RMSE: 0.06423343550447695
MAE : 0.04397544240339198
r2_score: 0.8157672116057979
Adjusted R2 score : 0.8131460133969373

```

The similarity in training and test data loss scores indicates that the model is not overfitting.

Mean Absolute Error (MAE) of 0.04: This implies an average deviation of 4% between the actual and predicted Chance of Admit values.

Root Mean Squared Error (RMSE) of 0.06: This indicates an average squared deviation of 6% between the predicted and actual values.

R² Score of 0.81: The model captures 81% of the variability in the data, showcasing its effectiveness in fitting the data.

Adjusted R²: This metric, an extension of R², adjusts for the number of predictors and illustrates how changes in the number of predictors impact the model's predictive accuracy.

Multicollinearity check using VIF Score

```
feature_names = ['GRE Score', 'TOEFL Score', 'University Rating',
'SOP', 'LOR', 'CGPA', 'Research'] # replace as needed
scaler = StandardScaler()
X_tr_scaled = scaler.fit_transform(X_train)
vif = pd.DataFrame()
X_t = pd.DataFrame(X_tr_scaled, columns=feature_names)
vif['Features'] = X_t.columns
vif['VIF'] = [variance_inflation_factor(X_t.values, i) for i in
range(X_t.shape[1])]
vif['VIF'] = round(vif['VIF'], 2)
vif = vif.sort_values(by='VIF', ascending=False)
vif

{"summary":{"name": "vif", "rows": 7, "fields": [
{"column": "Features", "properties": {
"dtype": "string", "num_unique_values": 7,
"samples": [
"CGPA", "GRE Score",
"LOR"
], "semantic_type": "",
"description": ""
}, {"column":
"VIF", "properties": {
"dtype": "number",
"std": 1.214164419712272, "min": 1.53, "max":
4.63, "num_unique_values": 7, "samples": [
4.63,
4.49,
2.03
], "semantic_type": "",
"description": ""
}
}
], "type": "dataframe", "variable_name": "vif"}
```

With all VIF scores below 5, there's little indication of pronounced multicollinearity issues

Residual analysis

```
y_predicted = lr.predict(X_test)
y_predicted.shape

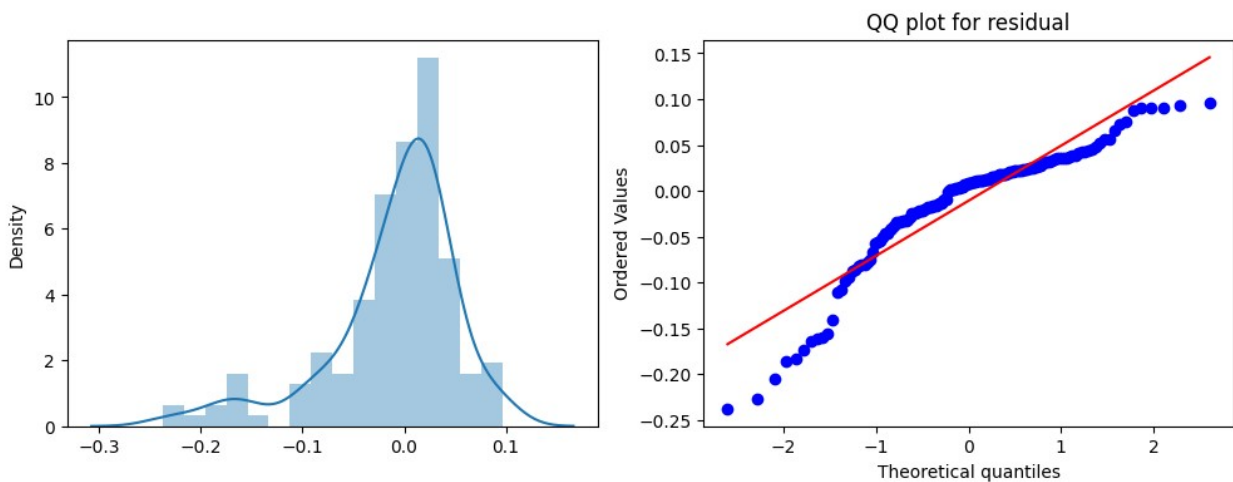
(150,)
```

```
#Mean of Residuals
residuals = y_test.values - y_predicted
print('Mean of Residuals: ', residuals.mean())
```

Mean of Residuals: -0.010793738256654513

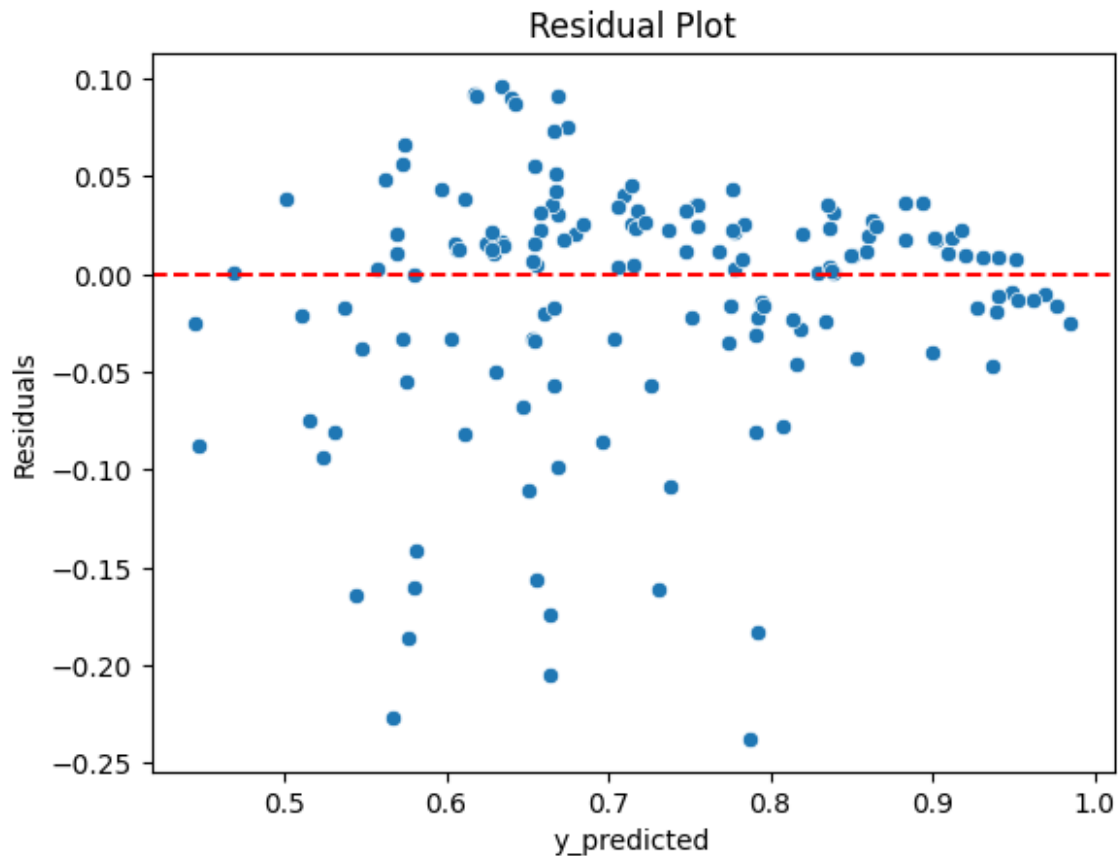
Since the mean of residuals is very close to 0, we can say that the model is unbiased

```
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
sns.distplot(residuals)
plt.subplot(1,2,2)
stats.probplot(residuals.reshape(-1, ), plot = plt)
plt.title('QQ plot for residual')
plt.show()
```



Test of homoscedasticity

```
# Test of homoscedasticity and plotting y_predicted and residual
sns.scatterplot(x = y_predicted.reshape((-1,)), y=residuals.reshape((-1,)))
plt.title('Residual Plot')
plt.xlabel('y_predicted')
plt.ylabel('Residuals')
plt.axhline(y=0, color='r', linestyle='--')
plt.show()
```



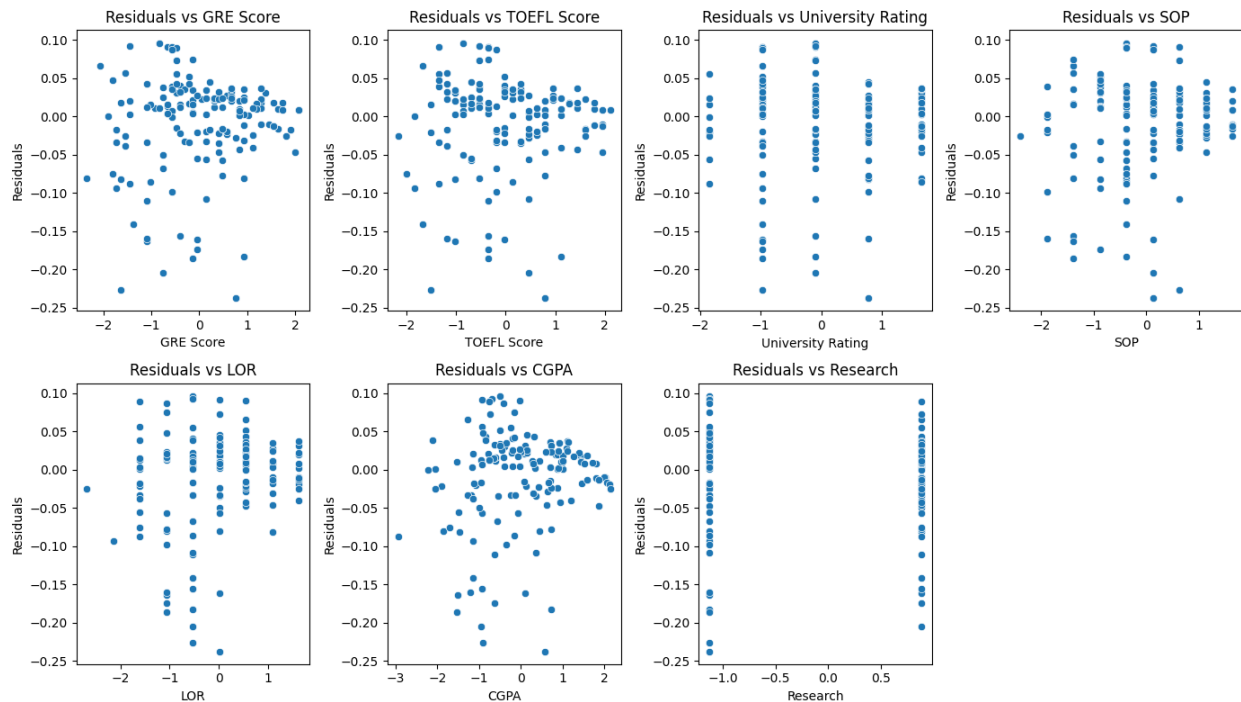
Since the residual plot shows no clear pattern or trend in residuals, we can conclude that linearity of variables exists.

```
# Scatterplot of residuals with each independent variable to check for
# homoscedasticity
feature_names = ['GRE Score', 'TOEFL Score', 'University Rating',
                 'SOP', 'LOR', 'CGPA', 'Research']

# Convert X_test (assumed to be a NumPy array) to DataFrame
X_test_df = pd.DataFrame(X_test, columns=feature_names)

# Plot residuals vs each feature
plt.figure(figsize=(14, 8))
for i, col in enumerate(feature_names, 1):
    ax = plt.subplot(2, 4, i)
    sns.scatterplot(x=X_test_df[col], y=residuals.reshape(-1,))
    plt.title(f'Residuals vs {col}')
    plt.xlabel(col)
    plt.ylabel('Residuals')

plt.tight_layout()
plt.show()
```



Model Regularisation

Ridge and Lasso Regression are regularization techniques designed to reduce the risk of overfitting in linear regression models. Both methods enhance model generalization by introducing a penalty term to the loss function, which discourages overly complex models:

Ridge Regression (L2 regularization) adds a penalty proportional to the square of the magnitude of the coefficients. This results in smaller, more evenly distributed weights, but typically does not eliminate features entirely.

Lasso Regression (L1 regularization) adds a penalty proportional to the absolute value of the coefficients. This can shrink some coefficients to exactly zero, effectively performing feature selection and simplifying the model.

By constraining the coefficient values, both methods aim to improve the model's performance on unseen data, balancing fit and complexity.

```
#Initialising instance of Ridge classes
model_ridge = Ridge()
# Fitting the models to training data
model_ridge.fit(X_train, y_train)

Ridge()

# Predicting values for train and test data
y_train_ridge = model_ridge.predict(X_train)
y_test_ridge = model_ridge.predict(X_test)

y_pred = model_ridge.predict(X_test)
print("MSE:", mean_squared_error(y_test, y_pred))
```

```

print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("MAE :", mean_absolute_error(y_test, y_pred) )
r2 = r2_score(y_test, y_pred)
print("R2 Score:", round(r2, 4))
print("Adjusted R2 Score:", round(AdjustedR2score(r2_score(y_test,
y_pred), len(y_test), X_test.shape[1]), 4))

```

```

MSE: 0.004132294408448722
RMSE: 0.06428292470360011
MAE : 0.04400979974255065
R2 Score: 0.8155
Adjusted R2 Score: 0.8064

```

```

y_predicted = model_ridge.predict(X_train)
y_predicted.shape

```

```

(350,)

```

```

#Mean of Residuals

```

```

residuals = y_train.values - y_predicted
print('Mean of Residuals: ', residuals.mean())

```

```

Mean of Residuals: 2.4107699963289113e-17

```

```

plt.figure(figsize=(12, 4))

```

```

plt.subplot(1, 2, 1)
sns.histplot(residuals, kde=True)
plt.title('Distribution of Residuals')

```

```

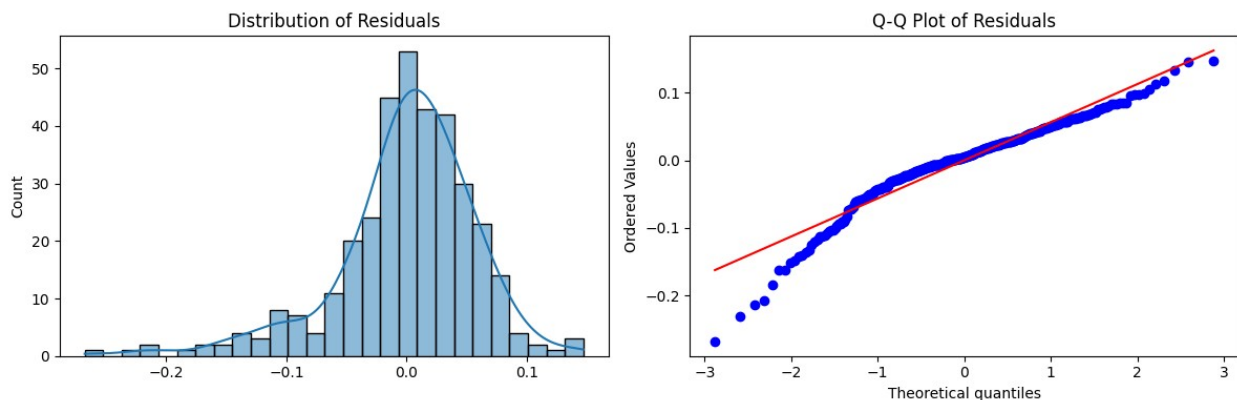
plt.subplot(1, 2, 2)
stats.probplot(residuals.reshape(-1, ), dist="norm", plot=plt)
plt.title('Q-Q Plot of Residuals')

```

```

plt.tight_layout()
plt.show()

```



Lasso Regression(L1 regularization)

```

model_lasso = Lasso()
model_lasso.fit(X_train, y_train)

y_train_lasso = model_lasso.predict(X_train)
y_test_lasso = model_lasso.predict(X_test)
y_pred = y_test_lasso

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
adj_r2 = AdjustedR2score(r2_score(y_test, y_pred), len(X_test),
X_test.shape[1])

print("MSE:", round(mse, 4))
print("RMSE:", round(rmse, 4))
print("MAE:", round(mae, 4))
print("R2 Score:", round(r2, 4))
print("Adjusted R2 Score:", round(adj_r2, 4))

MSE: 0.0225
RMSE: 0.1501
MAE: 0.1213
R2 Score: 0.8155
Adjusted R2 Score: -0.0555

y_predicted = model_lasso.predict(X_train)
residuals = y_train.values - y_predicted

print("Mean of Residuals:", round(residuals.mean(), 4))

plt.figure(figsize=(12, 4))

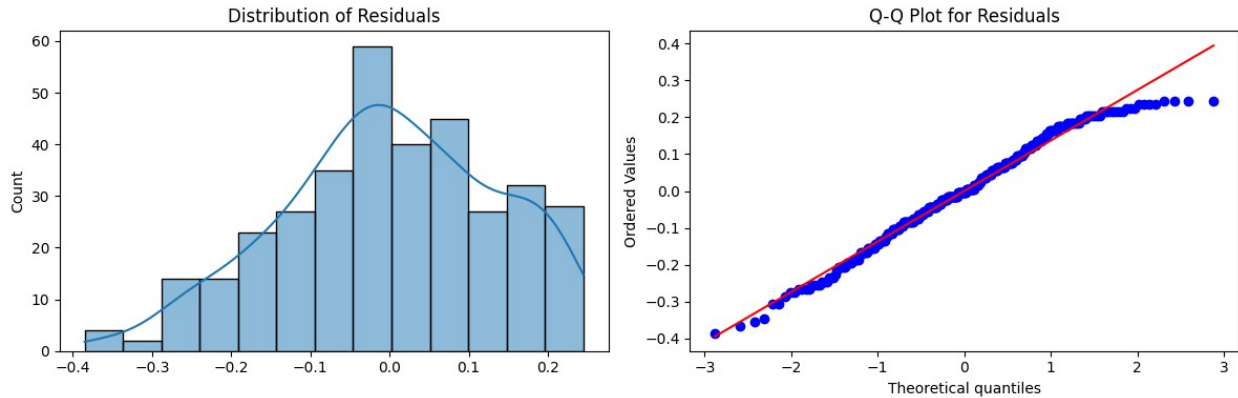
plt.subplot(1, 2, 1)
sns.histplot(residuals, kde=True)
plt.title('Distribution of Residuals')

plt.subplot(1, 2, 2)
stats.probplot(residuals.reshape(-1,), dist="norm", plot=plt)
plt.title('Q-Q Plot for Residuals')

plt.tight_layout()
plt.show()

Mean of Residuals: 0.0

```



ElasticNet (L1 + L2 Regularization)

```
model_enet = ElasticNet()
model_enet.fit(X_train, y_train)

y_train_enet = model_enet.predict(X_train)
y_test_enet = model_enet.predict(X_test)
y_pred = y_test_enet

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)

adj_r2 = AdjustedR2score(r2_score(y_test, y_pred), len(X_test),
X_test.shape[1])

print("MSE:", round(mse, 4))
print("RMSE:", round(rmse, 4))
print("MAE:", round(mae, 4))
print("R2 Score:", round(r2, 4))
print("Adjusted R2 Score:", round(adj_r2, 4))

y_predicted = model_enet.predict(X_train)
residuals = y_train.values - y_predicted

print("Mean of Residuals:", round(residuals.mean(), 4))

MSE: 0.0225
RMSE: 0.1501
MAE: 0.1213
R2 Score: 0.8155
Adjusted R2 Score: -0.0555
Mean of Residuals: 0.0

plt.figure(figsize=(12, 4))

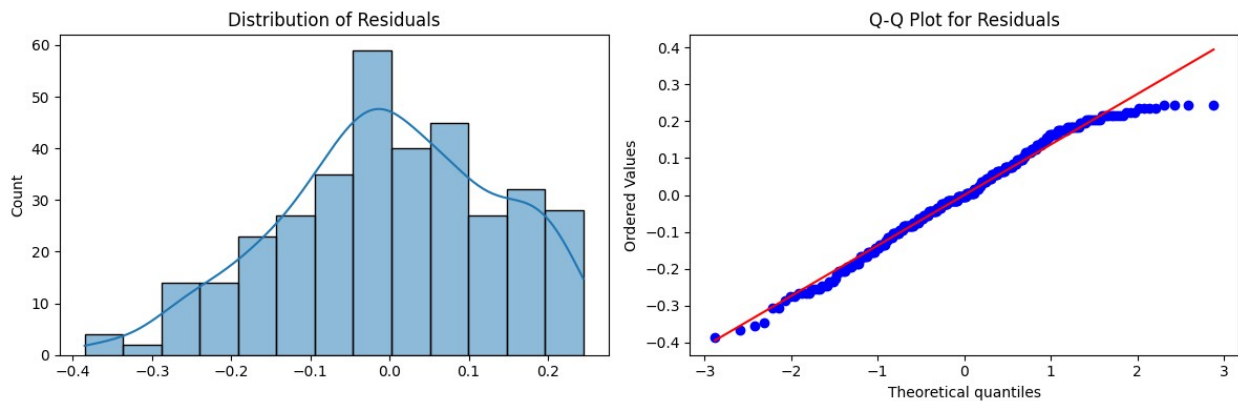
plt.subplot(1, 2, 1)
sns.histplot(residuals, kde=True)
```



```
plt.title('Distribution of Residuals')

plt.subplot(1, 2, 2)
stats.probplot(residuals.reshape(-1,), dist="norm", plot=plt)
plt.title('Q-Q Plot for Residuals')

plt.tight_layout()
plt.show()
```



Identifying the best Model

```
y_pred_train_lr = lr.predict(X_train)
y_pred_train_ridge = model_ridge.predict(X_train)
y_pred_train_lasso = model_lasso.predict(X_train)
y_pred_train_enet = model_enet.predict(X_train)

# Actual values
actual_values = y_train.values.reshape((-1,))

# List of predicted values for each model
predicted_values = [y_pred_train_lr, y_pred_train_ridge,
                    y_pred_train_lasso, y_pred_train_enet]

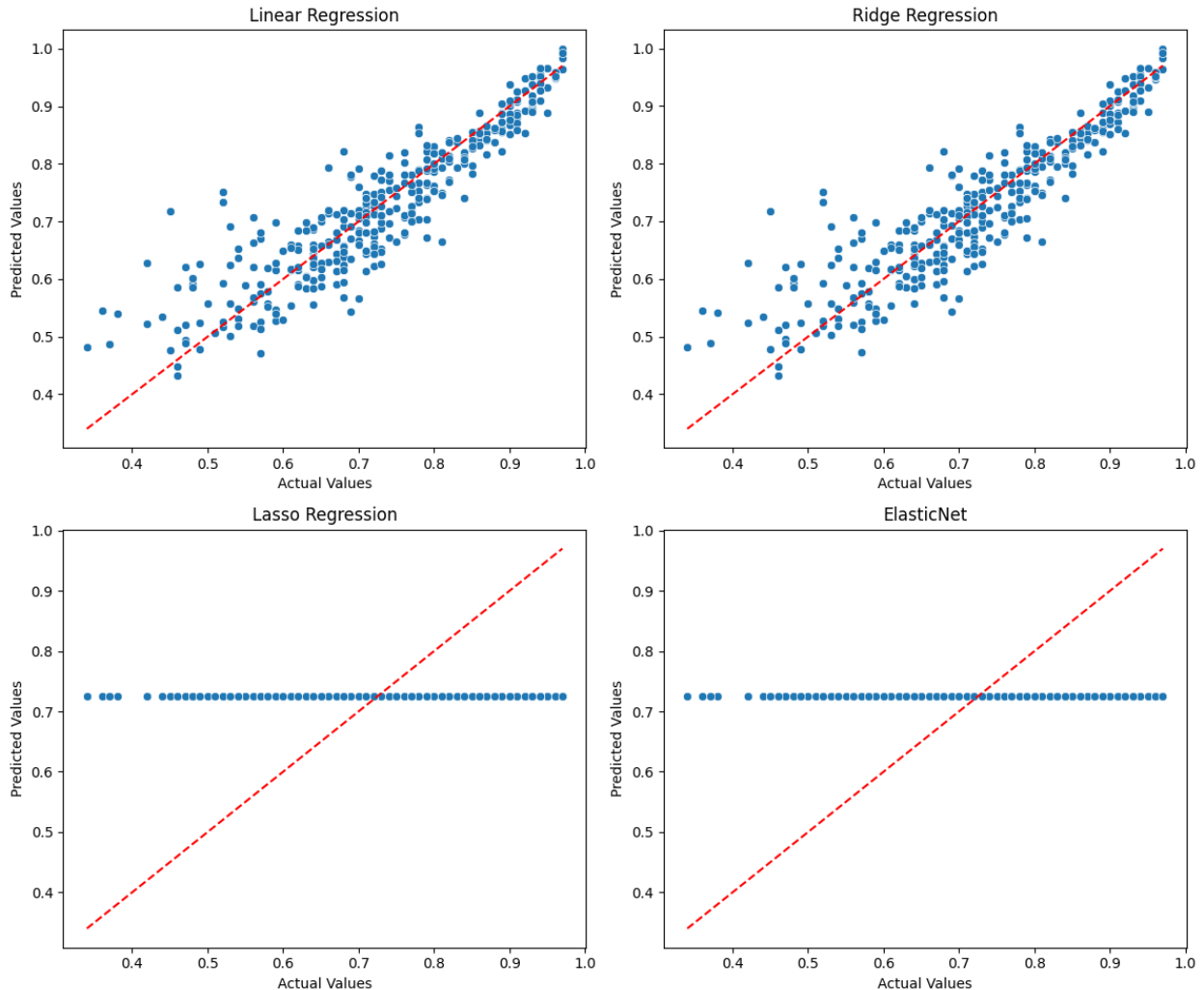
# List of model names
models = ['Linear Regression', 'Ridge Regression', 'Lasso Regression',
          'ElasticNet']

plt.figure(figsize=(12, 10))

# Loop over each model's predicted values to plot
for i, preds in enumerate(predicted_values, 1):
    ax = plt.subplot(2, 2, i)
    sns.scatterplot(x=actual_values, y=preds)
    plt.plot([min(actual_values), max(actual_values)],
             [min(actual_values), max(actual_values)], color='red', linestyle='--')
    plt.xlabel('Actual Values')
    plt.ylabel('Predicted Values')
```

```
plt.title(models[i-1])

plt.tight_layout()
plt.show()
```



Linear Regression and Ridge Regression yield comparable performance, while Lasso Regression and ElasticNet Regression underperform on both the training and test datasets.

```
# ElasticNet Model Metrics
y_pred_ElasticNet = model_enet.predict(X_test) # Use X_test instead
of x_test
ElasticNet_model_metrics = []
ElasticNet_model_metrics.append(mean_squared_error(y_test,
y_pred_ElasticNet)) # MSE
ElasticNet_model_metrics.append(np.sqrt(mean_squared_error(y_test,
y_pred_ElasticNet))) # RMSE
ElasticNet_model_metrics.append(mean_absolute_error(y_test,
y_pred_ElasticNet)) # MAE
```

```

ElasticNet_model_metrics.append(r2_score(y_test, y_pred_ElasticNet))
# R2 Score
ElasticNet_model_metrics.append(AdjustedR2score(r2_score(y_test,
y_pred_ElasticNet), len(X_test), X_test.shape[1])) # Adjusted R2

# Linear Regression Model Metrics
y_pred_LR = lr.predict(X_test) # Use X_test instead of x_test
LinearRegression_model_metrics = []
LinearRegression_model_metrics.append(mean_squared_error(y_test,
y_pred_LR)) # MSE
LinearRegression_model_metrics.append(np.sqrt(mean_squared_error(y_test,
y_pred_LR))) # RMSE
LinearRegression_model_metrics.append(mean_absolute_error(y_test,
y_pred_LR)) # MAE
LinearRegression_model_metrics.append(r2_score(y_test, y_pred_LR)) #
R2 Score
LinearRegression_model_metrics.append(AdjustedR2score(r2_score(y_test,
y_pred_LR), len(X_test), X_test.shape[1])) # Adjusted R2

# Ridge Model Metrics
y_pred_Ridge = model_ridge.predict(X_test) # Use X_test instead of
x_test
RidgeModel_model_metrics = []
RidgeModel_model_metrics.append(mean_squared_error(y_test,
y_pred_Ridge)) # MSE
RidgeModel_model_metrics.append(np.sqrt(mean_squared_error(y_test,
y_pred_Ridge))) # RMSE
RidgeModel_model_metrics.append(mean_absolute_error(y_test,
y_pred_Ridge)) # MAE
RidgeModel_model_metrics.append(r2_score(y_test, y_pred_Ridge)) # R2
Score
RidgeModel_model_metrics.append(AdjustedR2score(r2_score(y_test,
y_pred_Ridge), len(X_test), X_test.shape[1])) # Adjusted R2

# Lasso Model Metrics
y_pred_Lasso = model_lasso.predict(X_test) # Use X_test instead of
x_test
LassoModel_model_metrics = []
LassoModel_model_metrics.append(mean_squared_error(y_test,
y_pred_Lasso)) # MSE
LassoModel_model_metrics.append(np.sqrt(mean_squared_error(y_test,
y_pred_Lasso))) # RMSE
LassoModel_model_metrics.append(mean_absolute_error(y_test,
y_pred_Lasso)) # MAE
LassoModel_model_metrics.append(r2_score(y_test, y_pred_Lasso)) # R2
Score
LassoModel_model_metrics.append(AdjustedR2score(r2_score(y_test,
y_pred_Lasso), len(X_test), X_test.shape[1])) # Adjusted R2

# Creating a DataFrame to display the metrics

```

```

metrics_df = pd.DataFrame([LinearRegression_model_metrics,
LassoModel_model_metrics, RidgeModel_model_metrics,
ElasticNet_model_metrics],
                           columns=['MSE', 'RMSE', 'MAE', 'R2 Score',
'Adjusted R2'],
                           index=['Linear Regression', 'Lasso
Regression', 'Ridge Regression', 'ElasticNet Regression'])

# Displaying the metrics DataFrame
print(metrics_df)

```

	MSE	RMSE	MAE	R2 Score	
Adjusted R2					
Linear Regression	0.004126	0.064233	0.043975	0.815767	
0.806685					
Lasso Regression	0.022528	0.150094	0.121331	-0.005940	-
0.055528					
Ridge Regression	0.004132	0.064283	0.044010	0.815483	
0.806387					
ElasticNet Regression	0.022528	0.150094	0.121331	-0.005940	-
0.055528					

Observations:

The Chance of Admit distribution is left-skewed.

Exam scores (CGPA, GRE, TOEFL) show a strong positive correlation with the chance of admit, and there is also significant correlation between these variables.

Categorical variables like University Rating, Research, SOP, and LOR exhibit an upward trend concerning the chances of admit.

The first column was observed as a unique row identifier and was dropped as it was not necessary for model building.

There were no null values in the data.

No significant outliers were found in the data.

Chance of Admission (target variable) and GRE Score (independent feature) are nearly normally distributed.

Actionable Insights & Recommendations:

Augmenting the Model with Additional Features:

Given the strong correlation among exam scores (CGPA, GRE, TOEFL), it is recommended to include more independent variables to improve the model's predictive accuracy.

Increasing Awareness of CGPA and Research Capabilities:

Educational institutions should organize seminars to raise awareness about the importance of CGPA and research capabilities, which significantly influence the chances of admission.

Support for CGPA and Research Enhancement:

Educational institutions can help students improve not only their CGPA but also guide them in writing strong Letters of Recommendation (LOR) and Statements of Purpose (SOP). This support can improve their chances of admission to top universities.

Enhancing GRE Scores and Writing Skills:

Similarly, institutions can assist students in improving their GRE scores and in crafting strong LORs and SOPs, which can boost their chances of admission.

University Rating as a Predictor:

While university rating can serve as a good predictor, it has some outliers. It should be considered with caution when building predictive models.

Handling SOP, LOR, and Outliers:

While SOP and LOR are useful predictors, their weights may be less significant compared to exam scores. Additionally, outliers in these variables should be carefully handled during model building to ensure accurate predictions.